

CONVERSATION-LEVEL CYBERBULLYING DETECTION
USING LARGE LANGUAGE MODELS:
PROMPTING STRATEGIES, METADATA
AUGMENTATION, AND PARAMETER-EFFICIENT
FINE-TUNING

by

MANRAJ BOPARAI
URN: 6820105

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Diptesh Kanojia

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Manraj Boparai
May 2026

© Copyright Manraj Boparai, May 2026

Abstract

Cyberbullying detection at the conversation level requires reasoning over a full user-pair exchange rather than classifying individual messages in isolation. This project evaluates four open-weight instruction-tuned large language models: Llama 3.2-3B-Instruct, Mistral 7B-Instruct-v0.3, Qwen 3.5-9B, and Gemma 3-27B-IT, on a conversation-level cyberbullying dataset designed to incorporate four aspects of cyberbullying: aggression, repetition, peeriness, and intent to harm. The evaluation covers eight prompting configurations, including zero-shot, few-shot, and metadata-augmented variants, and three fine-tuning configurations on Llama 3.2-3B-Instruct.

The fine-tuning experiments compare LoRA against LoRMA+, a recent multiplicative low-rank adaptation method, under matched target modules, rank, scaling factor, training configuration, and base model. This project implements LoRMA+ as a Hugging Face-compatible module wrapper around the selected attention projection layers. During implementation, a discrepancy was identified between the initialisation described in the LoRMA+ paper and the initialisation used in its official reference implementation; both variants are evaluated under otherwise identical conditions.

Under matched hyperparameters, LoRA achieves a macro F1 score of 0.7437 on the test set, while LoRMA+ with the paper-described initialisation achieves 0.7285 and LoRMA+ with the reference implementation initialisation achieves 0.5564. All three fine-tuned configurations exceed the strongest prompting result, where Gemma 3-27B-IT with all three metadata fields reaches a macro F1 score of 0.6263. The 0.1721 macro F1 gap between the two LoRMA+ variants shows that the initialisation difference between the published method and its reference implementation has a substantial empirical effect on this task. Across the prompting configurations, the dominant failure mode is over-prediction of the minority class, with models often flagging non-cyberbullying conversations as cyberbullying.

The project also implements a parameterised experimental framework that replaces many near-duplicate scripts with a single command-line entry point, while ensuring that LoRA and LoRMA+ are compared under matched training conditions.

Acknowledgements

I would like to begin by thanking Waheguru Ji for giving me the strength and motivation to continue through both difficulties and challenges. My faith has been a source of comfort, focus, and motivation, helping me stay committed to my goals and complete this dissertation.

I am especially thankful to my grandfather, Bhajan Singh, for always pushing me educationally and encouraging me to keep working towards my goals. I am also deeply grateful to my mother, Paramjit Kaur, and to the rest of my family for their continued support, belief, and encouragement throughout my time at university.

I would also like to thank my supervisor, Dr Diptesh Kanojia, for his guidance, support, and valuable advice throughout this project. His feedback helped me approach the work with more confidence and direction.

Finally, I want to acknowledge myself for continuing to push forward, even during difficult moments, and for working hard to complete this dissertation.

Declaration of Generative AI Use

I acknowledge the use of generative AI (ChatGPT, OpenAI) during the preparation of this dissertation. It was used to support proofreading, improve clarity and structure in draft sections, provide feedback on readability, and refine the wording of the classification prompt used in the experiments. It was not used to generate the experimental design, implement the code, run the experiments, produce the results, or draw the conclusions. All AI-assisted suggestions were reviewed, edited, and approved by me before inclusion.

Contents

1	Introduction	7
1.1	Problem	7
1.2	Why Conversation-Level Detection Is Harder	8
1.3	Approach	9
1.4	Aims and Objectives	9
1.5	Summary of Contributions	10
2	Literature Review	11
2.1	Cyberbullying and Its Detection	11
2.1.1	Defining the Problem	11
2.1.2	Traditional Approaches	12
2.1.3	Deep Learning and Transformers	12
2.1.4	Conversation-Level Detection	12
2.1.5	The Gap Addressed	13
2.2	The Ejaz et al. Dataset	13
2.2.1	Provenance and Naming	13
2.2.2	Semi-Synthetic Construction	14
2.2.3	Implications for Interpretation	14
2.3	Large Language Models for Text Classification	15

2.3.1	Instruction-Tuned Decoder-Only Models	15
2.3.2	Zero-Shot, Few-Shot, and In-Context Learning	15
2.3.3	LLMs Applied to Abuse Detection	15
2.3.4	Model Selection	16
2.4	Parameter-Efficient Fine-Tuning	16
2.4.1	Motivation	16
2.4.2	LoRA	16
2.4.3	LoRMA+	17
2.4.4	Other PEFT Methods	18
3	Methodology	19
3.1	Dataset and Preprocessing	19
3.1.1	Dataset	19
3.1.2	Preprocessing	21
3.2	Experimental Framework	21
3.3	Prompt Design	24
3.3.1	The Base Prompt	24
3.3.2	Chat-Template Structure	25
3.3.3	Few-Shot Examples	25
3.3.4	Metadata-Augmented Prompting	26
3.3.5	Metadata vs Demographic Field Choice	26
3.4	Models and Inference	27
3.4.1	Model Summary	27
3.4.2	Memory and Quantisation	29
3.4.3	Inference Settings	29
3.4.4	Output Parsing	30

3.4.5	Hardware	30
3.4.6	Tokenisation and Context	30
3.5	Classical Baseline	31
3.6	LoRA Fine-Tuning of Llama 3.2-3B-Instruct	31
3.6.1	Configuration	31
3.6.2	Precision and Memory	32
3.6.3	Training Setup	32
3.6.4	Evaluation	33
3.7	LoRMA+ Fine-Tuning of Llama 3.2-3B-Instruct	33
3.7.1	Reference Implementation and Scope of Adaptation	34
3.7.2	Mathematical Formulation	35
3.7.3	Weight-Space Forward Pass	36
3.7.4	The <code>LoRMAPlusLinear</code> Module	36
3.7.5	Module Replacement	37
3.7.6	Initialisation-Scheme Ablation	37
3.7.7	Matching LoRA for Fair Comparison	39
3.7.8	Parameter Saving and Loading	39
3.8	Evaluation Metrics	40
4	Results and Evaluation	41
4.1	Evaluation Methodology	41
4.1.1	Zero-Shot and Few-Shot Prompting	42
4.2	Metadata Augmentation Ablation	43
4.3	Fine-Tuning: LoRA vs LoRMA+	44
4.3.1	Implications	45
5	Discussion	46

5.1	Summary of Key Findings	46
5.2	Comparison with Related Work	47
5.3	Limitations	48
5.4	Future Work	49
5.5	Personal Reflection	50
6	Legal, Social, Ethical and Professional Issues	51
6.1	Ethical Approval	51
6.2	Data Ethics and Confidentiality	51
6.3	Social Responsibility and Dual Use	52
6.4	BCS Code of Conduct	53
6.5	Computer Misuse Act	54
6.6	United Nations Sustainable Development Goals	54
6.7	Intellectual Property	55
7	Conclusion	56
A	Supplementary Material	58
A.1	Base Prompt	58
A.2	Few-Shot Examples	59
A.3	Hyperparameter Tables	59
A.4	Full Prompting and Fine-Tuning Results	62

List of Tables

3.1	Class distribution in the train, validation, and test splits.	20
3.2	Models used in this project.	27
3.3	Architectural specifications relevant to context handling.	28
4.1	Zero-shot and few-shot macro F1 across the four models on the test set. For reference, the majority-class baseline achieves macro F1 of 0.4732 and the TF-IDF baseline achieves 0.7161.	42
4.2	Macro F1 across zero-shot strategies with and without metadata augmentation. .	43
4.3	Llama 3.2-3B-Instruct fine-tuning results compared to the three strongest Llama prompting configurations.	45
A.1	LoRA hyperparameters for Llama 3.2-3B-Instruct.	59
A.2	LoRMA+ hyperparameters following the paper-described initialisation in Bihany et al. (2025a).	60
A.3	LoRMA+ hyperparameters following the initialisation used in the official LoRMA+ reference code. This is identical to the paper-init configuration except for the two initialisation rows.	61
A.4	Full per-experiment results on the test set.	62

Chapter 1

Introduction

1.1 Problem

Cyberbullying is the use of electronic communication to repeatedly harass, threaten, or intimidate an individual. Unlike face-to-face bullying, online cyberbullying is not bounded by physical location or immediate social context, and harmful interactions may continue across online conversations. Cyberbullying has been described as an international public-health concern among adolescents, with prior work linking victimisation to depression, anxiety, loneliness, suicidal behaviour, and stress-related physical symptoms Nixon (2014). Automated detection is therefore an important research problem for online safety.

A key challenge is that cyberbullying must be distinguished from general aggression or isolated offensive language. Olweus (2013) emphasises the importance of maintaining distinctions between bullying victimisation and general victimisation, and between bullying perpetration and general aggression. This distinction is important for automated detection because a single hostile message may be harmful, but may not by itself provide enough evidence of cyberbullying.

The central research question is whether automated systems can judge a full conversation in a way that approximates the decision a careful human moderator would make. Cyberbullying is commonly understood as repeated and targeted harmful behaviour carried out through digital communication, rather than as a property of an isolated message alone Smith et al. (2008). This makes conversation-level detection important because the meaning of an individual message may depend on the surrounding exchange.

Moderation errors have different consequences. False positives can silence legitimate users and erode trust, while false negatives leave victims unprotected. For this reason, accuracy alone is not sufficient for evaluation, especially when the dataset is strongly imbalanced.

1.2 Why Conversation-Level Detection Is Harder

Many automated abuse-detection datasets frame the task at message level, and the classifier outputs a binary abusive/non-abusive or aggressive/non-aggressive label Davidson et al. (2017); Waseem and Hovy (2016); Founta et al. (2018). A single rude message is not necessarily bullying; two users arguing in turn is not necessarily bullying; repeated, one-sided personal attacks against the same target are closer to the concept of cyberbullying. A message-level classifier is limited because it does not observe the wider conversation.

Conversation-level detection reframes the task as whether an exchange, viewed as a whole, constitutes cyberbullying. This introduces four difficulties. The classifier must reason across many messages and track which participant is targeting which. The input is much longer than a single post and may span dozens of messages and thousands of tokens. The labelling task is more subjective. Conversation-level labels are less common because the labelling process requires judgement over whole interactions rather than isolated messages.

Earlier text-classification methods are limited in this setting. Bag-of-words and TF-IDF classifiers treat each message as a vector of token counts and do not directly capture the order of messages. Fine-tuned BERT-based classifiers Devlin et al. (2019) can process conversations by concatenating messages into one input, but this is limited by input length and does not explicitly model turn-taking.

The Ejaz et al. dataset Ejaz et al. (2024a) provides over nine thousand conversation-level cyberbullying labels, along with metadata for peerness, intent to harm, and aggressive-message count. It is therefore suitable for evaluating models on conversation-level cyberbullying detection, where the label depends on context across the wider exchange.

1.3 Approach

Modern instruction-tuned large language models offer a different approach to conversation-level cyberbullying detection. Their context windows can accommodate full conversations, and their instruction-following capabilities allow the classification rules to be described in the prompt rather than manually encoded as features.

This project evaluates four open-weight instruction-tuned models: Llama, Mistral, Qwen, and Gemma. The evaluation uses the Ejaz et al. dataset and covers prompting-based classification, metadata-augmented prompting, and parameter-efficient fine-tuning with LoRA Hu et al. (2022) and LoRMA+ Bihany et al. (2025a). The detailed experimental design is described in Chapter 3, and the results are analysed in Chapter 4.

A key design choice is that the task is kept at conversation level. Although the dataset includes per-message aggression labels, this project does not reformulate the task as message-level aggression classification. The aim is to evaluate whether models can recognise cyberbullying as a conversation-level phenomenon involving repeated, targeted, and harmful behaviour.

1.4 Aims and Objectives

The overall aim is to investigate whether large language models can perform conversation-level cyberbullying detection effectively, and to characterise the conditions under which they succeed or fail.

The specific objectives are:

1. To implement a systematic experimental framework capable of evaluating multiple LLMs under multiple prompting strategies on the same dataset with minimal code duplication.
2. To evaluate four open-weight instruction-tuned LLMs on the dataset using zero-shot and few-shot prompting and compare their performance.
3. To investigate, through a controlled ablation, whether injecting the dataset’s social-context metadata into the prompt improves detection performance.
4. To implement LoRA and LoRMA+ on Llama 3.2-3B-Instruct, including both initialisation

variants of LoRMA+, and to compare the three configurations under matched hyperparameters.

5. To implement LoRMA+ in a form compatible with Hugging Face models, including deriving the forward pass for the wrapper module and integrating it with the training and evaluation pipeline.
6. To critically evaluate the results, identifying where LLMs succeed, where they fail, and what these patterns reveal about the difficulty of conversation-level cyberbullying detection.

1.5 Summary of Contributions

The main contributions of this project are:

- A systematic experimental framework for evaluating four LLMs, eight prompting strategies, and three fine-tuning configurations on the same dataset. The framework uses a shared runner and supporting modules, reducing duplication compared with maintaining separate scripts for each experimental configuration.
- A custom implementation of LoRMA+ as a Hugging Face-compatible wrapper for existing linear layers, applied to Llama 3.2-3B-Instruct as a parameter-efficient adaptation method.
- A controlled ablation study of metadata-augmented prompting, examining the effect of peerness, intent to harm, aggressive-message count, and their combination across all four models.
- A matched comparison of LoRA against two LoRMA+ initialisation variants under the same target modules, rank, scaling parameter, training configuration, and base model. This comparison identifies a discrepancy between the LoRMA+ paper description and its official reference implementation, discussed in Section 3.7.6.
- A comparative analysis of LLM behaviour on imbalanced conversation-level cyberbullying detection, focusing on over-flagging in prompting-based classification and under-detection in the reference-implementation LoRMA+ variant.

Chapter 2

Literature Review

2.1 Cyberbullying and Its Detection

2.1.1 Defining the Problem

Cyberbullying differs from general aggression because it usually involves repetition, targeting, and harm. Olweus (2013) emphasises the importance of distinguishing bullying from general aggression and victimisation, which is important for automated detection because a single hostile message may not provide enough evidence of bullying. Smith et al. (2008) examines the nature and impact of cyberbullying among secondary school pupils, making the paper relevant to this project’s focus on school-age cyberbullying scenarios.

These criteria are clear in theory but harder to apply in practice. A single message cannot establish repetition. Intent is not directly observable, and what constitutes harm may depend on social and conversational context. Labelling a single message as cyberbullying therefore risks treating cyberbullying as isolated aggression, rather than as a pattern across a wider interaction. Analysing the full conversation makes it possible to assess these features more directly than analysing isolated messages.

Smith et al. (2008) also discuss imbalance of power as an important feature of bullying, reflecting its role in traditional bullying research. In online contexts, power imbalance may arise from social status, group membership, coordination, or repeated targeting rather than physical dominance. The dataset used in this project, discussed in Section 2.2, includes a peerness field that partially captures the relationship between users.

2.1.2 Traditional Approaches

Early automated detection often treated the task as message-level classification using classical machine learning. Logistic regression and linear support vector machines applied to bag-of-words and TF-IDF representations remain common baselines Davidson et al. (2017); Waseem and Hovy (2016). These methods can perform reasonably well on tweet-style datasets, but they have important limitations. Because they rely heavily on word-level features, such as profanity or abusive keywords, they may wrongly flag harmless uses of strong language while missing abuse that is indirect, implied, or expressed without obvious offensive terms. They also do not model the structure of the conversation, so they treat abuse as something that can be judged from single messages rather than from patterns across a wider interaction.

2.1.3 Deep Learning and Transformers

Neural classifiers improved on earlier bag-of-words approaches by learning richer representations of text. CNNs and LSTMs using word embeddings improved earlier neural text classifiers. Transformer encoders, particularly BERT and its variants Devlin et al. (2019), later became widely used for text classification. Fine-tuned BERT-style classifiers can capture contextual information within a single input and are less dependent on exact keyword matches than classical methods.

However, these models are still often used as message-level classifiers in abuse-detection tasks. They can be applied to conversations by concatenating messages into a single input, but this approach is constrained by input length and does not explicitly represent conversational turn-taking. As a result, BERT-based classifiers may struggle when the label depends on repeated or targeted behaviour across multiple messages.

2.1.4 Conversation-Level Detection

Some prior work has explored cyberbullying detection at the conversation level. Rather than classifying each message independently, conversation-level approaches decide whether an interaction as a whole constitutes cyberbullying. This is important because repetition, targeting, and harmful intent may only become clear across multiple messages.

The Ejaz et al. dataset Ejaz et al. (2024a) addresses this problem by providing conversation-

level labels and metadata for peerness, intent to harm, and aggressive-message count. However, modern open-weight LLMs remain under-explored on this dataset, especially under controlled prompting, metadata-augmentation, and fine-tuning settings.

2.1.5 The Gap Addressed

This project addresses three main gaps. First, current-generation open-weight instruction-tuned LLMs have not been systematically benchmarked on this dataset. Second, where LLMs have been applied to abuse detection, the social-context metadata that some conversation-level datasets supply is rarely exploited; the Ejaz et al. dataset provides three such fields. Third, although LoRMA+ Bihany et al. (2025a) has been evaluated on standard NLU, NLG, and math QA benchmarks, its behaviour on conversation-level classification under strong class imbalance has not been reported. This project addresses all three gaps.

2.2 The Ejaz et al. Dataset

2.2.1 Provenance and Naming

This project uses *A Comprehensive Dataset for Automated Cyberbullying Detection*, introduced by Ejaz, Razi, and Choudhury Ejaz et al. (2024b,c). The experiments use the Hugging Face redistribution `surrey-nlp/Cyberbullying-Detection-CB2` NLP-AI Group, University of Surrey (2024), which provides pre-constructed train, validation, and test splits. This redistribution preserves the original dataset fields, conversation-level labels, and four-aspect labelling methodology.

The dataset represents cyberbullying through four constituent aspects Ejaz et al. (2024b,c): aggressive content, repetition, peerness, and intent to harm. This makes it suitable for conversation-level cyberbullying detection because it goes beyond labelling individual messages as aggressive or non-aggressive. A single rude or aggressive message may not be sufficient evidence of cyberbullying, whereas sustained targeted abuse is better captured by considering all four aspects.

2.2.2 Semi-Synthetic Construction

The dataset is semi-synthetic Ejaz et al. (2024c,a). Rather than collecting conversations directly, the authors combine synthetic user profiles with message text drawn from existing datasets.

Synthetic user pool. A set of 100 fictional users is generated procedurally, with each user assigned an age in the range $[8, 18]$, a gender, one of 15 fictional schools, and a grade derived from age.

Messages from existing datasets. Aggressive messages are drawn from Elsafoury’s Kaggle cyberbullying dataset Elsafoury (2020) and from the TRAC aggression-annotated Hindi-English code-mixed corpus of Kumar et al. (2018). Non-aggressive messages are drawn from the same sources, and the aggressive and non-aggressive message pools are balanced in size.

Synthetic communication log. For each sender-receiver user pair, the dataset generation process samples a random number of message events with random dates and times. Each message is sampled from either the aggressive or non-aggressive message pool, while preserving its original aggression label. The resulting communication log contains over 90,000 message events across approximately 9,500 unique user pairs.

The conversation-level cyberbullying label is then computed by applying thresholds to three derived quantities. A user pair (u_i, u_j) is labelled as cyberbullying if and only if

$$\text{Peerness}_{i,j} \geq 0.4 \wedge \text{IntentToHarm}_{i,j} \geq 0.4 \wedge \text{AggressiveCount}_{i,j} \geq 5. \quad (2.1)$$

The full release contains 1,021 cyberbullying conversations and 8,490 non-cyberbullying conversations Ejaz et al. (2024a).

2.2.3 Implications for Interpretation

The semi-synthetic construction has two consequences relevant to this project. The main advantage is that all four aspects of cyberbullying are explicitly quantified, which makes the metadata-augmentation ablation in Section 4.2 possible. The main limitation is that, although the individual messages are drawn from real datasets, the user pairings and conversation histories are synthetic. As a result, the cyberbullying label applies to a constructed user-pair interaction rather than to a naturally occurring conversation. This limitation is discussed further in Sec-

tion 5.3.

2.3 Large Language Models for Text Classification

2.3.1 Instruction-Tuned Decoder-Only Models

The four models used in this project are decoder-only transformers trained with a combination of next-token prediction and instruction-tuning Brown et al. (2020); Ouyang et al. (2022). The instruction-tuning phase gives the model the ability to follow natural-language task specifications without further task-specific gradient updates, which is what makes zero-shot classification with an instruction-tuned LLM feasible.

2.3.2 Zero-Shot, Few-Shot, and In-Context Learning

Zero-shot classification is performed by supplying a prompt that describes the classification criteria and the expected output format, followed by the input to classify. The model then generates an output beginning with the desired label. Few-shot classification extends this by adding labelled demonstrations to the prompt before the test input Brown et al. (2020). The model is shown several examples of the task being performed correctly, then asked to classify a new input. Few-shot prompting can improve performance on tasks with unusual output formats or subtle classification criteria, but the effect is not guaranteed.

2.3.3 LLMs Applied to Abuse Detection

Instruction-tuned models can be used for text classification by supplying the task definition and output format in the prompt Brown et al. (2020); Ouyang et al. (2022). This makes them attractive for online abuse detection because the classification policy can be changed by editing the prompt rather than retraining the classifier. However, prompted LLM classification can be sensitive to prompt wording, demonstration choice, and biases from pretraining data.

Published work on online abuse detection has historically focused on message-level hate-speech, offensive-language, and aggression datasets Davidson et al. (2017); Waseem and Hovy (2016); Founta et al. (2018); Kumar et al. (2018). These datasets are useful for studying abusive language, but they do not directly address conversation-level cyberbullying, where the label may

depend on repetition, targeting, and context across multiple messages. Few existing studies combine modern open-weight LLMs, conversation-level labels, and controlled metadata ablations.

2.3.4 Model Selection

Four models are evaluated, chosen to cover different parameter counts and open-weight model families: Llama 3.2-3B-Instruct Grattafiori et al. (2024), Mistral 7B-Instruct-v0.3 Jiang et al. (2023), Qwen 3.5-9B Qwen Team (2026a), and Gemma 3-27B-IT Gemma Team, Google DeepMind (2025); mradermacher (2025). The models range from 3.2B to 27B parameters, which allows the experiments to examine whether model scale affects performance on conversation-level cyberbullying detection. Three models are run using Hugging Face Transformers; Gemma 3-27B is run using `llama.cpp` Gerganov (2023) with 4-bit GGUF quantisation because the full-precision weights exceed the available VRAM.

2.4 Parameter-Efficient Fine-Tuning

2.4.1 Motivation

Full fine-tuning of large language models is expensive because it requires updating and storing gradients and optimiser states for all model parameters. Parameter-efficient fine-tuning (PEFT) methods address this problem by keeping the base weights frozen and training only a small set of additional parameters that modify the forward pass. In combination with 4-bit weight quantisation, QLoRA Dettmers et al. (2023) makes fine-tuning of large models feasible on substantially smaller hardware than full fine-tuning would require.

2.4.2 LoRA

Low-Rank Adaptation Hu et al. (2022) is the most widely used PEFT method. For a target linear layer with weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA introduces two matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ with $r \ll \min(d, k)$. The adapted forward pass computes

$$h = W_0 x + \frac{\alpha}{r} B A x, \quad (2.2)$$

equivalent to $W' = W_0 + (\alpha/r)BA$. The update BA is an additive low-rank correction. At

initialisation, A is Kaiming-sampled and B is zero, so $W' = W_0$ initially. Only A and B receive gradients. LoRA has a rank budget equal to r : the update can move W' only within a rank- r subspace.

2.4.3 LoRMA+

LoRMA (Low-Rank Multiplicative Adaptation) was proposed by Bihany, Patel, and Modi and published in the Findings of ACL 2025 Bihany et al. (2025a). The core idea is to replace LoRA’s additive update with a multiplicative one. In its base form, LoRMA computes $W' = BA \cdot W_0$, which is rank- r and restricts the rank of BAW_0 to at most r unless r is large. Two remedies are proposed in the paper; this project follows LoRMA+, the additive identity inflation variant:

$$W' = \left(I + \frac{\alpha}{r}BA\right)W_0 = W_0 + \frac{\alpha}{r}BAW_0. \quad (2.3)$$

The update is nearly full-rank, with rank at least $d - r$, where $r \ll d$, while the trainable parameters remain only A and B . The authors report faster convergence than LoRA Bihany et al. (2025a).

A practical concern with LoRMA+ is that a direct implementation materialises the full product BAW_0 as a $d \times k$ matrix. This is straightforward for unquantised weights but less straightforward when W_0 is stored in quantised form. Section 3.7.3 describes how this project handles the unquantised case; extending LoRMA+ to quantised base models would require an alternative implementation.

A second practical concern, identified during implementation, is that the initialisation described in the LoRMA+ paper and the initialisation used in the official reference implementation are different. Bihany et al. (2025a) state that LoRMA+ should be initialised with A randomly initialised and $B = 0$, matching the LoRA convention of a random input-side matrix and a zero output-side matrix. The official reference implementation instead initialises $A = 0$ and samples B using Kaiming-uniform initialisation. Both choices preserve $W' = W_0$ at step zero because $BAW_0 = 0$ in either case, but they may affect early optimisation differently. Sections 3.7.6 and 4.3 evaluate this difference empirically.

2.4.4 Other PEFT Methods

Other PEFT methods exist, including prefix-tuning and prompt-tuning Li and Liang (2021), adapter methods Houlsby et al. (2019), and IA³ Liu et al. (2022), but are not evaluated here. LoRA and LoRMA+ are the focus because they provide a direct additive–multiplicative comparison, with both methods applying low-rank adaptations to the same target linear layers.

Chapter 3

Methodology

This chapter describes the methodology used to evaluate large language models for conversation-level cyberbullying detection. It first describes the dataset and preprocessing pipeline, then explains the experimental framework, prompt design, model inference setup, fine-tuning procedure, LoRMA+ implementation, and evaluation metrics.

3.1 Dataset and Preprocessing

3.1.1 Dataset

This project uses *A Comprehensive Dataset for Automated Cyberbullying Detection*, introduced by Ejaz, Razi, and Choudhury Ejaz et al. (2024c) and released on Mendeley Data Ejaz et al. (2024a). For the experiments, the dataset is loaded through the Surrey-NLP Hugging Face redistribution `surrey-nlp/Cyberbullying-Detection-CB2` NLP-AI Group, University of Surrey (2024), a processed binary-label redistribution of the original Mendeley release that provides predefined train, validation, and test splits. The dataset source, original publication, and labelling process are discussed in Section 2.2; this section focuses on its properties relevant to model input, prompting, fine-tuning, and evaluation.

The dataset is loaded via `load_dataset("surrey-nlp/Cyberbullying-Detection-CB2")`. Each row corresponds to one user pair and contains the complete chronologically ordered message history between the two users in the `conversation` field. The fields used are `conversation`, the JSON-encoded message list; `label`, the binary conversation-level cyberbullying label; and the

three social-context metadata fields `peerness`, `intent_to_harm`, and `aggressive_count`. The demographic fields (`user1_age`, `user1_gender`, `user1_grade`, and the corresponding `user2_*` fields) and the identifier and summary fields (`user1_id`, `user2_id`, `total_messages`) are not used; the reasons for excluding the demographic fields are discussed in Section 3.3.5.

As discussed in Section 2.2, each `conversation` field represents the message history between user pairs rather than a single isolated exchange. This matters because the classifier is asked to make one conversation-level decision from a message history that may contain several distinct interactions between the same two users at different times. The model must judge whether the overall user-pair interaction constitutes cyberbullying as conceptually defined: sustained, targeted, repeated, and intentional harm rather than whether any individual message is offensive or aggressive in isolation. The class distribution of the supplied splits is shown in Table 3.1.

Table 3.1: Class distribution in the train, validation, and test splits.

Split	Label 0	Label 1	Total	Label 1 %
Train	6,389	744	7,133	10.4%
Validation	1,062	127	1,189	10.7%
Test	1,068	121	1,189	10.2%

The dataset is highly imbalanced, with label 1 accounting for only around 10% of each split. Accuracy alone is therefore not a sufficient measure of model performance: a naive classifier that always predicts label 0 would achieve approximately 90% accuracy but only 0.4732 macro F1 on the test set, failing to identify any cyberbullying conversations. This motivates the choice of macro F1 as the primary evaluation metric, since macro F1 gives equal weight to both classes rather than being dominated by the majority class.

The imbalance also influenced the prompt and parsing design. The base prompt instructs models to output 0 when the evidence for cyberbullying is unclear, reducing over-prediction of the minority class from isolated aggressive messages. During fine-tuning, no class re-weighting was applied, so that LoRA and LoRMA+ could be compared under the same original training distribution.

3.1.2 Preprocessing

Each row is processed into a standardised text representation so it can be passed as model input. The same pipeline is used for all four models and all eight prompting strategies.

The `conversation` field is parsed from its JSON string representation into an ordered list of message objects. The message-level aggression labels are deliberately excluded from the model input, because these labels would give the model direct information about which messages are already considered aggressive, making the conversation-level classification task easier than intended. The aim is to evaluate whether models can infer cyberbullying from the natural-language conversation content rather than from pre-computed annotations.

After extraction, messages are concatenated into a single conversation block, with each message placed on a separate line in chronological order. Speaker identifiers are not added, since they are not included in the source conversations.

Two truncation rules are used: a maximum of 50 messages (`MAX_MESSAGES = 50`) and a maximum of 30,000 characters (`MAX_CONV_CHARS = 30000`). The longest conversation in the dataset contains 40 messages and 26,881 characters, so no truncation occurs under this limit. For the Gemma GGUF backend, the character limit is reduced to 12,000 to match its smaller configured context window, as discussed in Section 3.4.6. Under this reduced cap, 236 conversations (2.5% of the dataset) are truncated, with a trailing ellipsis added before inference.

For the prompt variants that use social-context metadata, a metadata block is constructed from the `peerness`, `intent_to_harm`, and `aggressive_count` fields immediately before the conversation text. Missing values are represented as the literal string `unknown` rather than imputed. The full injection format is described in Section 3.3.4.

3.2 Experimental Framework

The project covers 32 prompting-based configurations, corresponding to four models evaluated across eight prompting strategies, and three fine-tuning configurations on Llama 3.2-3B-Instruct. The overall structure of the framework, showing how the shared preprocessing and evaluation stages connect the three experimental pipelines (prompting, classical baseline, and fine-tuning), is summarised in Figure 3.1.

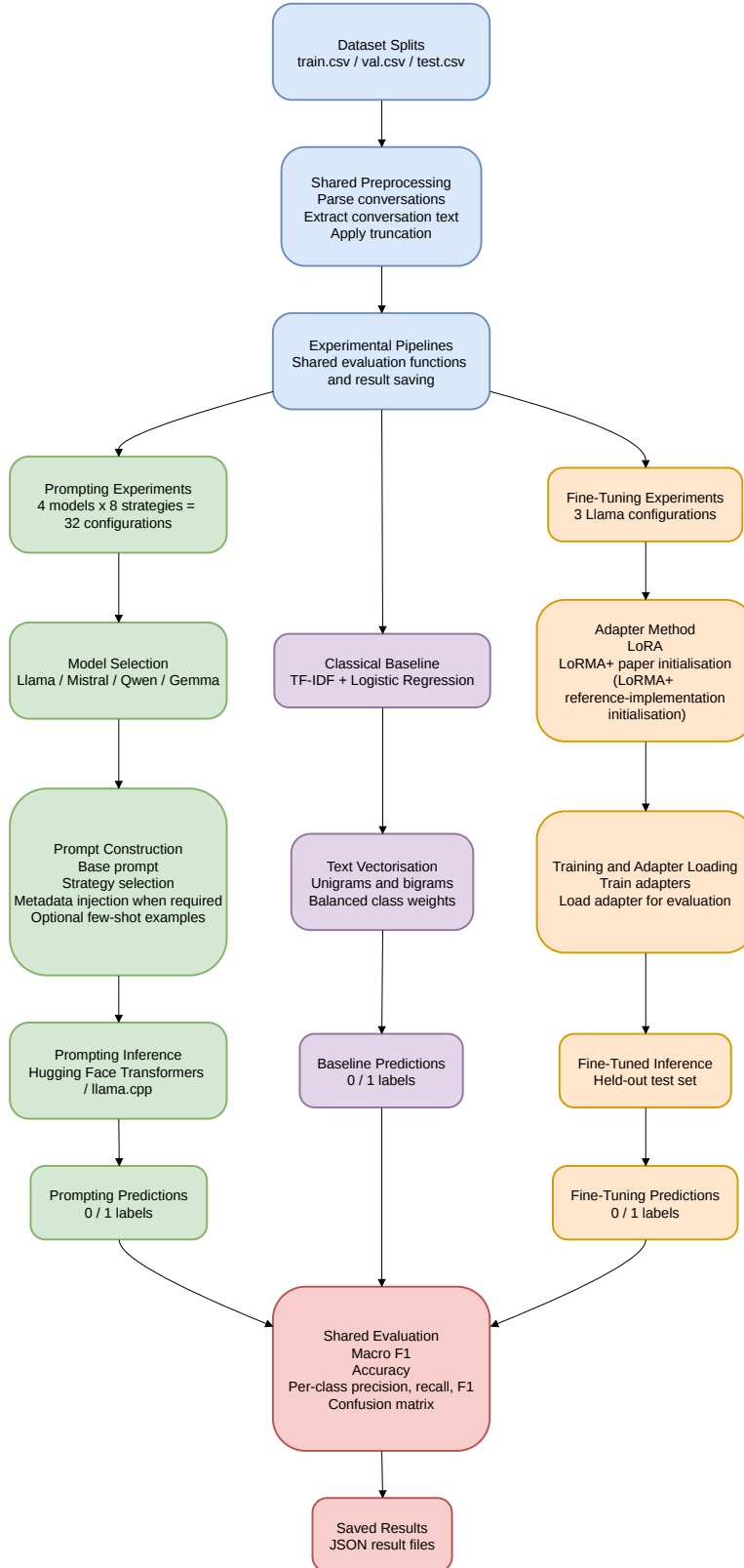


Figure 3.1: Overview of the experimental framework. The shared dataset and preprocessing stage feeds into three pipelines: prompting, classical baseline, and fine-tuning. All pipelines produce binary predictions and are evaluated using the same shared metrics.

The framework was initially implemented as a separate script per configuration. Three examples are retained in `src/legacy/`: a zero-shot baseline, a few-shot variant, and a combined few-shot-with-metadata variant. These show how each strategy variant required nearly duplicate files despite differing only in which metadata fields and examples were included in the prompt. Extending this approach across all four models and eight strategies would have produced large amounts of duplicated code and made the codebase harder to maintain, since changes to model loading, conversation truncation, or prompt wording would need to be applied manually across many scripts.

For this reason the prompting experiments were refactored into a single parameterised runner:

```
python run_experiment.py --model <name> --strategy <name>
```

The runner dispatches on the model name to the corresponding inference backend, either Hugging Face Transformers or `llama.cpp`, and on the strategy name to construct the corresponding prompt. The model configuration is centralised in `MODELS` in `config.py`, while the prompting logic is handled in `prompts.py`, which selects the metadata fields and few-shot examples according to the strategy name.

The framework consists of six Python files totalling approximately 700 lines. Implementing the same functionality as one script per configuration would have required approximately 6,200 lines (extrapolated from the average line count of the retained legacy scripts), an 87% reduction. This makes the implementation easier to review, easier to extend, and less likely to contain accidental differences between experimental conditions.

The fine-tuning experiments are kept in separate scripts because their workflow differs from the inference-only prompting loop. Fine-tuning requires dataset preparation, training, checkpoint saving, adapter loading, and held-out evaluation. Three fine-tuning scripts are used: `llama_lora.py` for the standard LoRA baseline, `llama_lorma_plus_a0_bkaiming.py` for LoRMA+ using the reference-implementation initialisation, and `llama_lorma_plus_akaiming_b0.py` for LoRMA+ using the paper-described initialisation. Although these scripts are separate from the prompting runner, they import `extract_conversation_text` and `BASE_PROMPT` from the shared framework and use the same evaluation functions as the prompting experiments. This ensures that the fine-tuned models receive inputs in the same format and are scored using the same metrics as the prompting configurations. The implementation details of LoRA and LoRMA+ are described in Sections 3.6 and 3.7.

3.3 Prompt Design

3.3.1 The Base Prompt

All configurations use a shared base prompt as the system message. The prompt’s wording was informed by the dataset description provided by Ejaz, Razi, and Choudhury Ejaz et al. (2024c), which frames cyberbullying as multi-faceted, involving aggression, repetition, peerness, and intent to harm. The same description highlights a limitation of existing message-level datasets, where a single aggressive message may be treated as cyberbullying without accounting for repetition, peerness, or harmful intent. This motivated the prompt’s emphasis on conversation-level patterns rather than isolated offensive words.

Given the dataset’s strong class imbalance (Section 3.1.1) and the labelling convention under which the positive class requires sustained and targeted patterns rather than single aggressive messages, the prompt was designed to reduce the risk of over-predicting the positive class.¹ The prompt lists examples that should remain label 0 (isolated insults, profanity, short arguments, teasing, unclear targeting) and instructs the model to return 0 when the evidence is weak, mixed, or uncertain. The wording was kept general rather than model-specific so that the same prompt could be used consistently across all four instruction-tuned models. The full text of the base prompt is reproduced in Appendix A.1.

Four design choices are deliberate. First, the definition is given in terms of behavioural pattern rather than vocabulary, which prevents the model from collapsing the task into profanity detection. Second, the negative class is enumerated explicitly. Third, the model is biased toward 0 under uncertainty, which is suitable for an imbalanced binary classification task where the positive class should require clear evidence of targeted cyberbullying. Fourth, the output is constrained to a single character to improve parsing reliability and bound inference cost.

¹Generative AI (ChatGPT, OpenAI) was used to suggest wording refinements during development of the classification prompt. The final prompt, the four-point structure, the negative-class enumeration, and all experimental decisions were determined and verified by the author.

3.3.2 Chat-Template Structure

For the three Hugging Face-backed models, the prompt is constructed as a list of role-content messages using **system**, **user**, and **assistant** roles, and passed to each tokenizer’s `apply_chat_template` method Hugging Face (2026a). This converts the structured messages into the model-specific prompt format, including the special markers that separate the system, user, and assistant parts. Using each tokenizer’s own chat template avoids manually writing separate prompt formats for each model: different instruction-tuned models use different role markers, and a format written for one model may not be suitable for another.

For few-shot strategies, each demonstration is added as a **user** message containing the example conversation, followed by an **assistant** message containing the correct label. This matches the question-answer pattern the model is expected to produce at test time.

For Gemma, which uses the `llama.cpp` backend with a GGUF model file, the Hugging Face chat-template mechanism is not used. Instead, a plain-text prompt with equivalent content is constructed and passed directly to the GGUF model. This difference is noted because the two backends receive the prompt in slightly different formats, which is a source of variation when comparing results across models.

3.3.3 Few-Shot Examples

The few-shot variants prepend four labelled examples to the conversation being classified. Two examples are labelled 1 and two are labelled 0, giving the prompt a balanced demonstration set despite the dataset’s class imbalance. The examples were drawn from the training split and chosen to cover four distinct subtypes: sustained personal attack (label 1), repeated personal insults (label 1), heated disagreement without bullying (label 0), and isolated offensive messages without sustained targeting (label 0). Each example is formatted as a **user** message containing the conversation text followed by an **assistant** message containing only the label character, matching the structure used at test time. The full text of the examples is not reproduced in the main report because it contains lengthy abusive content; the examples are included in the submitted source file `src/framework/few_shot_examples.py`.

3.3.4 Metadata-Augmented Prompting

Six metadata-augmented strategies inject one or all three of the dataset’s metadata fields into the prompt: `zero_shot_peer`, `zero_shot_intent`, `zero_shot_aggressive`, `zero_shot_meta`, `few_shot_peer`, and `few_shot_meta`. The `*_meta` strategies include all three metadata fields together.

The choice of fields follows from the four-aspect definition of cyberbullying underlying the dataset, discussed in Section 2.2. Three aspects are explicitly represented as numeric metadata and correspond directly to the dataset’s labelling criteria: `aggressive_count` captures aggression, `peeriness` reflects the peer relationship between participants, and `intent_to_harm` measures inferred harmful intent. The fourth aspect, repetition, is not provided as a standalone metadata field but is represented through `total_messages` and through the conversation text itself. `total_messages` was not used as a separate prompt field because it measures the overall length of the user-pair interaction rather than repeated aggression specifically.

Evaluating each field individually and in combination tests whether the dataset-derived signals improve classification, and whether any single field contributes most of the effect. Metadata is added immediately before the conversation text as a short key-value block:

```
Peerness: 0.847
Intent-to-harm: 0.120
Aggressive-count: 2

Conversation:
<conversation text>
```

Missing values are represented as `unknown` rather than imputed. The six-way ablation results are presented in Section 4.2.

3.3.5 Metadata vs Demographic Field Choice

The dataset also contains demographic fields for both users, including `user*_age`, `user*_gender`, and `user*_grade`. These fields were deliberately excluded from the prompt-based experiments for two reasons.

First, this project focuses on behavioural evidence in the conversation rather than user characteristics. The synthetic user pool spans ages 8–18, and including age, gender, or grade information

in the prompt could encourage the model to use demographic attributes when making a moderation decision. The model should identify cyberbullying from evidence in the conversation, such as repeated aggression, targeting, and harmful intent, rather than from who the users are. This issue is discussed further in Section 6.3.

Second, the metadata ablation is intended to test fields that directly relate to the dataset’s cyberbullying framing: `aggressive_count`, `peeriness`, and `intent_to_harm`. Including demographic fields would shift the focus of the experiment toward whether the model treats different age, gender, or grade groups differently, which is separate from this project’s aim of testing whether metadata linked to the cyberbullying definition improves classification.

The possible use of demographic fields is noted in Section 5.4 as a direction for future work on bias analysis.

3.4 Models and Inference

3.4.1 Model Summary

Four open-weight instruction-tuned LLMs are evaluated, ranging from 3.2B to 27B parameters. The model identifiers, backends, and precision settings are summarised in Table 3.2. Architectural properties relevant to context handling are summarised in Table 3.3. The model identifiers, parameter counts, vocabulary sizes, context lengths, and attention mechanisms are taken from the official model cards, configuration files, and technical documentation Meta AI (2024b,a); Mistral AI (2024b,a); Hugging Face (2026b); Qwen Team (2026a,b); Google DeepMind (2025). The backend and precision columns reflect their implementation in this project, including the GGUF deployment used for Gemma mradermacher (2025).

Table 3.2: Models used in this project.

Name	Identifier	Params	Backend	Precision
Llama	meta-llama/Llama-3.2-3B-Instruct	3.2B	HF Transformers	FP16
Mistral	mistralai/Mistral-7B-Instruct-v0.3	7.2B	HF Transformers	FP16
Qwen	Qwen/Qwen3.5-9B	9.0B	HF Transformers	4-bit bitsandbytes
Gemma	google/gemma-3-27b-it	27B	llama.cpp	IQ4_XS GGUF

Three models are run through the Hugging Face Transformers backend, while Gemma 3-27B is

Table 3.3: Architectural specifications relevant to context handling.

Property	Llama 3.2-3B	Mistral 7B	Qwen 3.5-9B	Gemma 3-27B
Parameters	3.2B	7.2B	9.0B	27B
Vocabulary size	128,256	32,768	248,320	262,144
Max context	131,072	32,768	262,144	131,072
Attention pattern	GQA	Sliding-window / GQA	Hybrid linear/full	Local/global

run through `llama.cpp` using a quantised GGUF model file. The difference is due to hardware constraints, discussed in Section 3.4.2: Gemma is substantially larger than the other models, and the quantised GGUF version makes it practical to run within the available GPU memory. This also affects prompt formatting, since the Hugging Face models use tokenizer chat templates whilst the Gemma GGUF backend receives a single plain-text prompt, as described in Section 3.3.2.

The four models differ in scale, tokenisation, and context handling, all of which may affect performance on conversation-level classification. Parameter count ranges from 3.2B (Llama 3.2) to 27B (Gemma 3), which affects how well each model captures longer conversation patterns. Vocabulary size varies from 32,768 (Mistral) to 248,320 (Qwen); larger vocabularies split words into fewer tokens, reducing the token count of long conversations and affecting how much context fits into the model input. Attention mechanisms also differ: Mistral uses sliding-window attention with a 4,096-token window, so distant parts of a long conversation may not directly attend to each other; Gemma alternates local and global attention layers; Qwen uses a hybrid linear/full attention scheme; and Llama uses grouped-query attention. Conversations are capped at approximately 30,000 characters, or around 7,500–8,500 tokens, so context length is not a strict limitation for most models, but architectural differences remain relevant because each model combines information across long conversations differently.

During inference, context length was not a limiting factor for most models. For Gemma 3-27B the constraint came from the GGUF runtime rather than the model’s official context window: a reduced context setting and character cap were used to ensure the input fit within the available runtime configuration, as discussed in Section 3.4.6.

3.4.2 Memory and Quantisation

Gemma 3-27B at full precision would require approximately 54 GB of VRAM for weights alone Google DeepMind (2025), which exceeds the 20 GB available on the RTX 4000 Ada GPU used in this project. For this reason, Gemma was loaded from a community-distributed GGUF redistribution of the official model weights: `mradermacher/gemma-3-27b-it-GGUF`, using the file `gemma-3-27b-it.IQ4_XS.gguf`. `IQ4_XS` is an importance-aware 4-bit quantisation scheme supported by `llama.cpp` that preserves the more important parts of the weight representation, helping to reduce the quality loss associated with low-bit quantisation. This reduced the weight memory requirements from approximately 54 GB at full precision to approximately 14 GB, making Gemma 3-27B practical to run on the available hardware.

Qwen 3.5-9B is loaded in 4-bit via `bitsandbytes` for the same memory reason. Llama 3.2-3B and Mistral 7B fit within the available GPU memory for prompting and were loaded in FP16.

Fine-tuning is performed only on Llama 3.2-3B-Instruct, as described in Sections 3.6–3.7. The model fits within the 20 GB GPU memory budget in bfloat16 during training, including the additional memory required for gradients, optimiser state, and activations, so no quantisation was required for the fine-tuning experiments.

3.4.3 Inference Settings

All inference uses greedy decoding. For the Hugging Face-backed models this is implemented with `do_sample=False`; for the GGUF backend the equivalent deterministic setting is used with `temperature=0.0`. Greedy decoding is used because the model is expected to generate either 0 or 1, and this makes the results deterministic across repeated runs.

A maximum of six new tokens is allowed for each prediction (`max_new_tokens=6`). Although the prompt asks the model to output a single character, allowing six tokens makes the parser tolerant of minor formatting deviations such as leading whitespace or a newline before the label. The small token budget also keeps inference cost low and discourages unnecessary explanatory output.

For Qwen, extended thinking mode is disabled using `enable_thinking=False`. The task requires only a single binary label, and chain-of-thought style output would increase token usage and make output parsing less reliable. Disabling this mode keeps Qwen’s output format closer to

that of the other models.

3.4.4 Output Parsing

After generation, the model output is converted into a binary prediction. The generated text is stripped of leading and trailing whitespace. If the output begins with 1 the prediction is recorded as label 1; if it begins with 0 it is recorded as label 0.

Additional checks are included because the models may occasionally produce minor formatting artefacts, such as a newline, a trailing period, or extra text before the label. The parser also checks each line of the generated output and accepts lines containing only 0 or 1, or a label preceded by a dot. As a final fallback, a regular expression identifies any line that consists only of a valid label.

If no valid label is found, the prediction defaults to 0. This choice reflects the class imbalance in the dataset and the prompt instruction to return 0 when the evidence for cyberbullying is weak or uncertain. Parse failures were negligible across the configurations, based on inspection of the outputs.

3.4.5 Hardware

All experiments were conducted on University of Surrey Otter lab machines. The main experiments used the `otter133`–`otter162` machines, which each have a single NVIDIA RTX 4000 Ada Generation GPU with 20 GB of VRAM. Model weights, datasets, and Hugging Face cache files were stored on the university Linux machines’ scratch storage at `/scratch/mb02997`.

3.4.6 Tokenisation and Context

The character cap `MAX_CONV_CHARS = 30000` was chosen to fit under every model’s context window with headroom for the prompt and its components. The largest prompt configuration (few-shot with metadata on Llama 3.2-3B-Instruct) totals approximately 9,300 tokens, well within the smallest 32,768-token context window. During fine-tuning, `MAX_SEQ_LENGTH = 6144` is sufficient for the prompt and conversation text in nearly all training examples; the longest examples may be truncated by the tokenizer, affecting only the tail of the length distribution.

3.5 Classical Baseline

A TF-IDF and logistic regression classifier is included as a non-neural baseline. The literature review in Section 2.1.2 notes that classical bag-of-words classifiers remain a common reference point for abuse detection. Including one here provides an anchor against which the LLM prompting and fine-tuning results can be compared, making it possible to quantify the gain attributable to large language models over a simple, well-understood method.

Each conversation is converted to text using the same `extract_conversation_text` function used in the prompting and fine-tuning pipelines, so the classifier receives the same input representation as the LLMs. The features are unigrams and bigrams with a vocabulary cap of 50,000, weighted by sublinear term frequency and inverse document frequency. Classification uses logistic regression with L2 regularisation, $C = 1.0$, balanced class weights to compensate for the approximately 9:1 class imbalance, and a maximum of 1,000 optimisation iterations. The model is trained on the training split and evaluated on the held-out test set using the same metrics described in Section 3.8.

Two design choices are noted. First, no hyperparameter tuning was performed. The baseline is included as a reference point rather than as a competitive system, and its purpose is to test whether the more expensive LLM methods improve on a simple non-neural reference. Second, balanced class weights are used for the logistic regression baseline but not in the LoRA or LoRMA+ fine-tuning runs, which follow the LoRMA+ reference implementation in not applying class re-weighting. This asymmetry favours the baseline and is therefore a conservative comparison for the fine-tuned configurations.

3.6 LoRA Fine-Tuning of Llama 3.2-3B-Instruct

3.6.1 Configuration

LoRA fine-tuning was performed on Llama 3.2-3B-Instruct using the Hugging Face PEFT implementation. The adapter was configured with rank $r = 16$, scaling parameter $\alpha = 32$, dropout 0.05, target modules `q_proj`, `k_proj`, `v_proj`, and `o_proj`, and task type `CAUSAL_LM`.

The LoRA settings were chosen as a balanced PEFT configuration following the original LoRA formulation and QLoRA’s reported hyperparameter ranges Hu et al. (2022); Dettmers et al.

(2023). Rank $r = 16$ keeps the number of trainable parameters small while allowing a useful low-rank update; $\alpha = 32$ gives a scaling ratio of $\alpha/r = 2$; dropout 0.05 provides modest regularisation. The target modules `q_proj`, `k_proj`, `v_proj`, and `o_proj` are the attention projection matrices Vaswani et al. (2017) that the original LoRA work adapts in transformer-based models Hu et al. (2022). The same target modules are used for the LoRMA+ experiments so that the comparison adapts the same parts of the model.

Llama 3.2-3B contains 28 transformer blocks. LoRA is applied to four attention projection modules in each block, giving 112 adapted projection layers in total. With rank $r = 16$, the LoRA configuration produces approximately 9.18M trainable parameters, representing around 0.28% of the full base model.

3.6.2 Precision and Memory

The base model was loaded in `bfloat16` with no weight quantisation. At full precision, the 3.2B-parameter base model requires approximately 6.4 GB of memory for weights alone, which remained within the available 20 GB GPU memory budget once the additional memory for gradients, optimiser state, and activations was included.

Llama 3.2-3B-Instruct was the only model fine-tuned in this project. The larger models evaluated during prompting (Mistral, Qwen, and Gemma) would not fit for `bfloat16` fine-tuning on the available 20 GB GPU once optimiser states and activation memory are included. Fine-tuning those models would have required quantised PEFT methods such as QLoRA Dettmers et al. (2023), which is designed to reduce memory usage while preserving performance. However, using quantisation for some fine-tuning runs but not others would have introduced an additional variable into the LoRA-versus-LoRMA+ comparison. Restricting fine-tuning to Llama 3.2-3B-Instruct allowed all three configurations LoRA, paper-initialised LoRMA+, and code-initialised LoRMA+ to be compared under the same precision setting.

3.6.3 Training Setup

Training used TRL `SFTTrainer` with `SFTConfig` Hugging Face (2026c) and completion-only loss (`completion_only_loss=True`), so the loss was computed only on the label token rather than the full prompt. The model was trained for 2 epochs with learning rate 1×10^{-4} , per-device batch size 1, gradient accumulation over 8 steps, maximum sequence length 6,144 tokens, and `bfloat16`

precision. After applying the base prompt and Llama chat template, 4 of the 7,133 training conversations (0.06%) exceed 6,144 tokens (99th percentile 3,831 tokens; longest example 8,076 tokens), so truncation is not expected to influence the LoRA-versus-LoRMA+ comparison.

3.6.4 Evaluation

After training, the LoRA adapter was saved and reloaded on top of the same base model. The evaluation pipeline was kept identical to the zero-shot prompting pipeline: each test conversation was formatted using the shared base prompt, passed to the adapted model, and parsed as a binary prediction.

Metrics were computed using scikit-learn and included accuracy, macro F1, class-wise precision, class-wise recall, class-wise F1, and the confusion matrix. These results are reported in Section 4.3.

3.7 LoRMA+ Fine-Tuning of Llama 3.2-3B-Instruct

The LoRMA+ implementation in this project was developed with reference to the official LoRMA repository released by Bihany et al. Bihany et al. (2025b), and adapted so that LoRMA+ could be applied to Llama 3.2-3B-Instruct using bfloat16 base weights and the binary conversation-level classification task.

An ablation of the LoRMA+ initialisation scheme was also performed. This was motivated by a discrepancy identified during implementation: the reference implementation initialises the LoRMA+ matrices A and B in the opposite order to the method described in Section 3.2.2 of Bihany et al. Bihany et al. (2025a). An initial run using the reference-implementation initialisation produced substantially weaker results than the LoRA baseline, predicting mostly the majority class. The ablation compares the paper-described initialisation with the reference-implementation initialisation under otherwise identical training conditions, as discussed in Section 3.7.6.

3.7.1 Reference Implementation and Scope of Adaptation

Two files from the `NLG/Language-generation/src/` directory of the official LoRMA repository Bihany et al. (2025b) are used as reference: `model_lorma_plus.py` and `gpt2_ft_lorma_plus.py`. The repository also contains a LoRA baseline (`model.py`, `gpt2_ft.py`) and a LoRMA $_{\pi}$ variant (`model_lorma_pi.py`, `gpt2_ft_lorma_pi.py`); neither is used in this project. The LoRA baseline is not re-implemented because this project’s LoRA configuration uses the Hugging Face PEFT library instead of `loralib`. The LoRMA $_{\pi}$ variant uses permutation-based rank inflation rather than the additive identity inflation that defines LoRMA+, and is outside the scope of this project.

From `model_lorma_plus.py`, this project adopts the mathematical formulation of the LoRMA+ update. In the reference implementation, this appears inside `Attention.forward` as:

```
adapted_W_q = W_q + scaling * (lora_B_q @ (lora_A_q @ W_q))
query = F.linear(x, adapted_W_q, bias)
```

The reference implementation initialises `lora_A` to zero and `lora_B` with a Kaiming-uniform distribution. This differs from the initialisation described in Section 3.2.2 of Bihany et al. Bihany et al. (2025a), which describes the opposite convention. This project evaluates both initialisations under identical training conditions; see Section 3.7.6.

From `gpt2_ft_lorma_plus.py`, two utility functions were adapted and renamed for this project. The function `mark_only_lora_as_trainable` was renamed to `mark_only_lorma_as_trainable`, and `mult_lora_state_dict` was renamed to `lorma_state_dict`. Both functions identify LoRMA+ parameters by checking whether the parameter name contains `lora_A` or `lora_B`: the first freezes all other parameters during training, and the second extracts only the adapter weights for saving.

Beyond these adopted components, this project contributes a linear-layer wrapper class `LoRMAPlusLinear` that replaces the reference implementation’s approach of placing the LoRMA+ logic directly inside the attention block; a module-replacement utility `apply_lorma_plus` that attaches the wrapper to selected projection layers; an initialisation-scheme ablation comparing the paper-described and reference-implementation conventions; and a LoRMA+ training and evaluation pipeline that mirrors the LoRA baseline so that the comparison focuses on the adaptation method rather than on differences in training setup. Each of these components is described in the subsections that follow.

3.7.2 Mathematical Formulation

LoRMA+ modifies the original layer weights using a multiplicative low-rank update. For a frozen weight matrix W_0 , the adapted weight is defined as:

$$W' = W_0 + \frac{\alpha}{r} B A W_0. \quad (3.1)$$

Here, W_0 is the original weight matrix, while A and B are the trainable LoRMA+ matrices. The corresponding forward pass for an input vector x is:

$$h = W'x = W_0x + \frac{\alpha}{r} B A W_0x. \quad (3.2)$$

This differs from standard LoRA Hu et al. (2022), where the low-rank update is added directly to the base-model weight matrix:

$$W'_{\text{LoRA}} = W_0 + \frac{\alpha}{r} B A. \quad (3.3)$$

The difference is important because LoRMA+ applies the low-rank transformation through the existing weight matrix W_0 , whereas LoRA learns an additive update that is independent of W_0 .

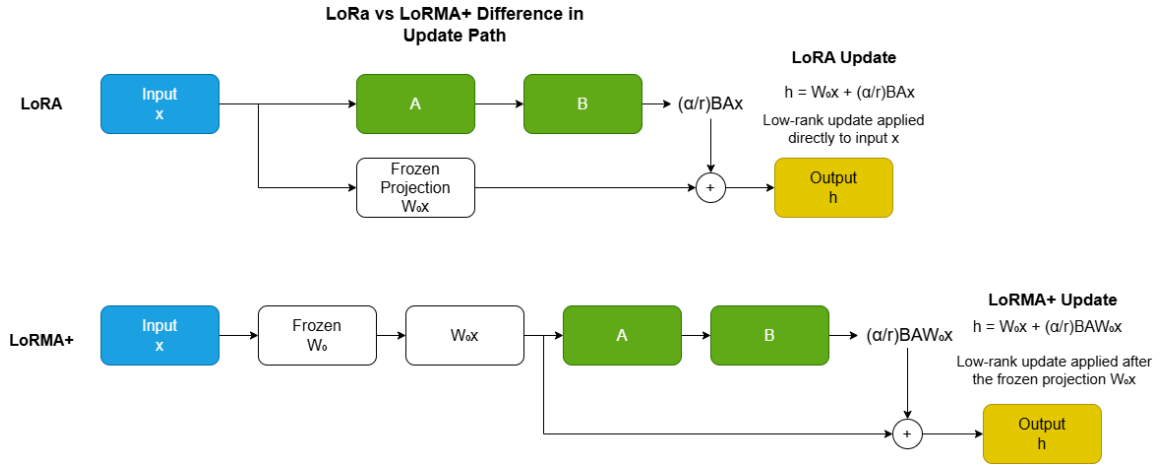


Figure 3.2: Difference in update path between standard LoRA and LoRMA+. LoRA applies the low-rank update directly to the input x , whereas LoRMA+ applies the low-rank update after the frozen projection W_0x .

Figure 3.2 illustrates the structural difference between the two update paths. In LoRA, the input x flows directly into the low-rank matrices A and B , and the resulting correction is added to the frozen projection W_0x . In LoRMA+, the frozen projection W_0x is computed first, and

the low-rank matrices then act on W_0x rather than on x . The LoRMA+ update is therefore conditioned on the existing weight matrix, which is the source of the multiplicative routing through W_0 .

At step zero, both initialisations produce $BA = 0$ and therefore $BAW_0 = 0$. The LoRMA+ update term contributes nothing to the forward pass, so the adapted model’s output equals the unmodified model output. The reference-implementation initialisation achieves this by setting $A = 0$ and initialising B with a Kaiming-uniform distribution; the paper-described initialisation achieves it by setting $B = 0$ and initialising A with a Kaiming-uniform distribution. Once training begins, the zero-initialised matrix receives gradient updates and stops being zero, allowing the LoRMA+ update to contribute to the forward pass. Both initialisations, and the difference between them, are discussed in Section 3.7.6.

3.7.3 Weight-Space Forward Pass

Since Llama 3.2-3B is loaded without weight quantisation, the base weight W_0 is directly accessible as an ordinary weight matrix. The weight-space form of the LoRMA+ update can therefore be computed directly:

$$W' = W_0 + \frac{\alpha}{r}B(AW_0), \quad h = W'x.$$

For the frozen weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the LoRMA+ matrices are shaped $A \in \mathbb{R}^{r \times d_{\text{out}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$. The product AW_0 has shape (r, d_{in}) , and $B(AW_0)$ has shape $(d_{\text{out}}, d_{\text{in}})$, matching W_0 . The forward pass is then a single linear operation through `F.linear(x, adapted_W, bias)`, where `adapted_W` is the resulting adapted weight matrix W' . This is the form used in all fine-tuning experiments in this project.

3.7.4 The LoRMAPlusLinear Module

In the reference implementation, LoRMA+ is built directly into the GPT-2 attention block in `model_lorma_plus.py`. In this project, LoRMA+ is implemented instead as a `nn.Module` wrapper around an existing linear layer. The original module is retained as `self.base_linear`, and the two LoRMA+ matrices A and B are registered as `nn.Parameter` attributes on the wrapper.

The forward pass for the weight-space variant is:

```

def forward(self, x):
    W = self.base_linear.weight
    bias = self.base_linear.bias
    A = self.lora_A.to(dtype=W.dtype, device=W.device)
    B = self.lora_B.to(dtype=W.dtype, device=W.device)
    adapted_W = W + self.scaling * (B @ (A @ W))
    return F.linear(x, adapted_W, bias)

```

The parameter tensors have shapes $A \in \mathbb{R}^{r \times d_{\text{out}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$, matching the layout used in the reference implementation. The scaling factor α/r is computed once during initialisation and stored as a module attribute. Registering `lora_A` and `lora_B` as `nn.Parameter` attributes ensures they receive gradients during training; their values are extracted at save time as described in Section 3.7.8.

3.7.5 Module Replacement

After loading the frozen Llama 3.2-3B-Instruct model, `apply_lorma_plus` traverses `model.named_modules()` and replaces every module whose path ends with one of the target projection names: `q_proj`, `k_proj`, `v_proj`, or `o_proj`. Each matched module is replaced with a `LoRMAPlusLinear` wrapper around the original linear layer. The original projection weight remains frozen, while the wrapper adds trainable `lora_A` and `lora_B` matrices.

Llama 3.2-3B has 28 transformer blocks and four target modules per block, giving 112 wrapped projection layers. Across these layers, the LoRMA+ implementation introduces 7,340,032 trainable parameters (approximately 7.34M). This is lower than the matched LoRA configuration’s 9.18M trainable parameters because both LoRMA+ matrices depend on d_{out} , while LoRA’s input-side matrix depends on d_{in} ; the difference is most visible in the key and value projections, where $d_{\text{in}} = 3072$ but $d_{\text{out}} = 1024$ due to Llama’s grouped-query attention. This parameter-count difference is reported for transparency rather than corrected by changing the rank, since the rank, scaling factor, target modules, and training setup are otherwise matched.

3.7.6 Initialisation-Scheme Ablation

LoRA’s standard initialisation Hu et al. (2022) sets the matrix nearest the input to a Kaiming-uniform distribution and the matrix nearest the output to zero. This makes the adapted model identical to the base model at initialisation while still allowing gradients to flow into the zero-

initialised output-side matrix during the first optimisation steps.

The LoRMA+ paper text (Bihany et al., 2025a, Section 3.2.2) follows the same convention for LoRMA+, stating that B should be set to zero while A is randomly initialised. Under the paper’s update rule,

$$W' = W_0 + \frac{\alpha}{r} B A W_0,$$

A acts first on $W_0 x$, and B then acts on the result. The paper-described initialisation therefore sets A to Kaiming-uniform and B to zero, mirroring the standard LoRA convention of randomising the matrix that acts first in the adapter path and zeroing the matrix that produces the final adapter output.

The reference implementation released with the LoRMA+ paper uses the opposite convention. In `model_lorma_plus.py`, the relevant parameters are declared as follows:

```
self.lora_A_q = nn.Parameter(...new_zeros((r, n_state)))
self.lora_B_q = nn.Parameter(...new_zeros((n_state, r)))
nn.init.kaiming_uniform_(self.lora_B_q, a=math.sqrt(5))
```

This leaves A initialised to zero and applies Kaiming-uniform initialisation to B . An initial training run under this convention produced substantially weaker results than the LoRA baseline, which was unexpected given that Bihany et al. report LoRMA+ to perform comparably to LoRA on natural language generation benchmarks Bihany et al. (2025a). This motivated a controlled ablation of the two initialisation conventions, with both configurations trained for the same number of epochs and all other experimental settings kept fixed:

- **LoRMA+ (paper init):** A is Kaiming-uniform and $B = 0$, matching the initialisation described in Bihany et al. (Bihany et al., 2025a, Section 3.2.2).
- **LoRMA+ (code init):** $A = 0$ and B is Kaiming-uniform, matching the initialisation used in the official LoRMA+ reference repository Bihany et al. (2025b).

Both schemes give $BAW_0 = 0$ at initialisation, so both adapted models initially implement the same function as the frozen base model, $W' = W_0$. The ablation therefore isolates the effect of which low-rank factor carries the random component during the first optimisation steps. Results are reported in Section 4.3.

3.7.7 Matching LoRA for Fair Comparison

The three fine-tuning configurations: LoRA, LoRMA+ with paper-described initialisation, and LoRMA+ with reference-implementation initialisation were matched in every respect except the adaptation method and, for the two LoRMA+ runs, the initialisation scheme. All three used Llama 3.2-3B-Instruct as the base model with unquantised bfloat16 weights and compute; the same target modules (`q_proj`, `k_proj`, `v_proj`, `o_proj`); rank $r = 16$; scaling parameter $\alpha = 32$; TRL `SFTTrainer` with completion-only loss; the AdamW optimiser; learning rate 1×10^{-4} ; per-device batch size 1; gradient accumulation over 8 steps; 2 training epochs; maximum sequence length 6,144 tokens; and the same prompt format and tokenisation pipeline used throughout the project.

Two residual differences are noted for transparency. First, the LoRA configuration applies dropout 0.05 on the low-rank path, while the LoRMA+ variants apply no dropout. Each setting follows its respective reference convention: removing dropout from LoRA would depart from the standard LoRA baseline, while adding dropout to LoRMA+ would introduce a hyperparameter not present in the reference implementation and would require its own tuning. Since the dropout rate is low, its effect on the comparison is expected to be limited, but the difference is noted as a remaining limitation.

Second, the saving procedure differs because LoRA is implemented through Hugging Face PEFT, whereas LoRMA+ is implemented as a custom module. LoRA uses PEFT-aware checkpoint serialisation with `save_strategy="epoch"`; the LoRMA+ variants use `save_strategy="no"` and save only the custom LoRMA+ parameters after training via `torch.save(lorma_state_dict(model), ...)`. This affects checkpoint handling only; the training loop, optimisation settings, evaluation pipeline, and model inputs are unchanged across the three fine-tuning configurations.

3.7.8 Parameter Saving and Loading

At the end of training, `lorma_state_dict(model)` extracts only parameters whose names contain `lora_A` or `lora_B`. These parameters are moved to CPU and written to disk using `torch.save`. The saved file therefore contains only the LoRMA+ adapter parameters, not the frozen base model weights. For each LoRMA+ configuration, this corresponds to approximately 7.34M parameters, or around 29 MB when stored in fp32.

At evaluation time, the base Llama 3.2-3B-Instruct model is freshly loaded in bfloat16, the same `LoRMAPlusLinear` wrappers are attached to the target modules, and the saved LoRMA+ state is loaded using `load_state_dict(..., strict=False)`. Non-strict loading is used because the saved file contains only the custom LoRMA+ parameters, while the full model state also includes the frozen pretrained base weights.

3.8 Evaluation Metrics

The same evaluation protocol is used across all 35 experimental configurations: the 32 prompting configurations described in Sections 3.3–3.4 and the three fine-tuning configurations described in Sections 3.6–3.7. The same protocol is also applied to the TF-IDF and logistic regression baseline described in Section 3.5. Each configuration is evaluated on the same 1,189-conversation test set, using the same prediction format and the same scikit-learn metric functions. This ensures that prompting, classical baseline, LoRA, and LoRMA+ results are directly comparable.

Macro F1 is used as the primary metric because the test set is strongly imbalanced, with label 1 representing only around 10% of the examples. Macro F1 gives equal weight to both classes rather than allowing the majority class to dominate the score. This matters because a classifier that always predicts label 0 would achieve approximately 0.898 accuracy but only 0.4732 macro F1.

Accuracy is also reported, but only as a secondary metric. Per-class precision, recall, and F1 are reported to show whether a model is missing cyberbullying cases or over-flagging non-cyberbullying ones. A confusion matrix is reported for each experiment to make the false-positive and false-negative trade-off directly visible.

Chapter 4

Results and Evaluation

4.1 Evaluation Methodology

All results in this chapter are computed on the fixed test set of 1,189 conversations, consisting of 1,068 non-cyberbullying examples labelled 0 and 121 cyberbullying examples labelled 1. The test set is therefore strongly imbalanced, with class 1 representing only approximately 10.2% of the samples.

Two reference points are used to contextualise the results. First, a majority-class predictor that always outputs label 0 achieves a macro F1 score of 0.4732 and an accuracy of 0.8982. This is not a trained model: its scores are computed directly from the test-set label distribution. It establishes the floor below which any classifier must be considered failing, and it illustrates why accuracy alone is not an adequate metric, a classifier that detects no cyberbullying cases achieves nearly 90% accuracy. Macro F1 is therefore used as the main comparison metric because it gives equal weight to both classes.

Second, a TF-IDF and logistic regression classifier, described in Section 3.5, provides a classical supervised-learning baseline. This model achieves a macro F1 score of 0.7161 and an accuracy of 0.8511 on the test set, with class-1 precision of 0.387 and class-1 recall of 0.793. Its confusion matrix is (TN, FP, FN, TP) = (916, 152, 25, 96).

Scores below the majority-class macro F1 baseline indicate that a configuration performs worse than always predicting the non-cyberbullying class. Scores below the TF-IDF baseline indicate that the configuration does not outperform a simple classical text classifier on this dataset. Both

reference points are included in the fine-tuning comparison in Table 4.3.

4.1.1 Zero-Shot and Few-Shot Prompting

Table 4.1 reports macro F1 scores under zero-shot and few-shot prompting without metadata. Three observations follow.

Table 4.1: Zero-shot and few-shot macro F1 across the four models on the test set. For reference, the majority-class baseline achieves macro F1 of 0.4732 and the TF-IDF baseline achieves 0.7161.

Model	Zero-shot	Few-shot
Llama	0.3948	0.3174
Mistral	0.4743	0.5481
Qwen	0.5257	0.4144
Gemma	0.5952	0.5113

First, model scale appears relevant but does not dominate. Gemma 3-27B is the strongest zero-shot performer, followed by Qwen, Mistral, and Llama. These differences cannot be explained by model size alone, however, because the models also differ in instruction tuning, tokenisation, architecture, and deployment precision. Llama is the only model whose zero-shot macro F1 falls below the majority-class baseline of 0.4732. Its confusion matrix $TN = 387$, $FP = 681$, $FN = 2$, $TP = 119$ shows that it over-predicts label 1, flagging 800 of the 1,189 test conversations.

Second, few-shot prompting does not consistently improve performance. For Mistral, few-shot prompting improves macro F1 from 0.4743 to 0.5481. For Llama, Qwen, and Gemma, however, it reduces macro F1 by 0.0774, 0.1113, and 0.0839 respectively. Inspection of the confusion matrices shows that, in every few-shot configuration except Mistral, the model shifts further toward predicting label 1. These configurations achieve class-1 recall between 0.95 and 1.00, but class-0 recall falls as low as 0.25.

One likely explanation is a mismatch between the few-shot examples and the test set. The few-shot set is balanced, with two positive and two negative examples, while the test set is approximately 9:1 in favour of label 0. Because the few-shot examples are balanced, the model may assume that positive and negative cases are equally common, causing it to over-predict label 1 on the imbalanced test set. Mistral is the exception because its zero-shot baseline already over-

predicts label 1, so the examples move it closer to the correct decision balance rather than further away.

Third, the dominant error pattern across all eight zero-shot and few-shot configurations is over-flagging. Class-1 recall ranges from 0.78 to 1.00, while class-1 precision ranges only from 0.13 to 0.24. Even the strongest configuration, Gemma zero-shot with a macro F1 score of 0.5952, flags 397 conversations and is correct on only 95 of them. Notably, no prompting configuration across any of the four models exceeds the TF-IDF baseline of 0.7161 macro F1. This shows that the prompt-only models are generally sensitive to cyberbullying cues but tend to over-predict the cyberbullying class on this imbalanced dataset, and that they do not outperform a well-tuned classical text classifier on this task.

4.2 Metadata Augmentation Ablation

Table 4.2 reports macro F1 across the four zero-shot metadata-augmented configurations alongside the no-metadata zero-shot baseline reproduced from Table 4.1. The best score for each model is shown in bold.

Table 4.2: Macro F1 across zero-shot strategies with and without metadata augmentation.

Model	None	Peerness	Intent	Aggressive	All three
Llama	0.3948	0.4027	0.4367	0.4139	0.4150
Mistral	0.4743	0.4435	0.4360	0.4528	0.4216
Qwen	0.5257	0.5478	0.5945	0.4904	0.5988
Gemma	0.5952	0.5840	0.6033	0.5682	0.6263

Metadata augmentation is beneficial overall for the stronger models. Gemma improves from 0.5952 to 0.6263 when all three metadata fields are included, a gain of 0.0311. Qwen improves more substantially, from 0.5257 to 0.5988, a gain of 0.0731. For both models, intent-to-harm is the most useful individual metadata field, while combining all three fields gives the strongest overall result.

For the weaker models, metadata is less useful and can be harmful. Llama improves slightly under some metadata conditions, with its best score rising from 0.3948 to 0.4367 when intent-to-harm is included; however, its absolute performance remains below the majority-class baseline

of 0.4732. Mistral loses performance under every metadata configuration, with its best zero-shot result obtained using no metadata.

This pattern suggests that social-context metadata is useful only when the model can combine it effectively with the conversation text. For weaker models, the additional metadata may increase prompt complexity without improving classification performance.

A useful pattern in the test set is worth noting: 608 of the 1,189 conversations have missing peerness and 147 have peerness in the lowest quartile (≤ 0.37). All of these conversations are labelled 0. Of the remaining 434 conversations with peerness above this threshold, 121 are labelled 1, meaning that approximately 28% of higher-peerness conversations are cyberbullying cases. Peerness therefore acts as a strong filter for the positive label. This reflects the threshold rule from the dataset construction (Section 2.2) and helps explain why metadata augmentation has its strongest effect when peerness is combined with intent-to-harm and aggressive-message count.

4.3 Fine-Tuning: LoRA vs LoRMA+

Table 4.3 compares LoRA, LoRMA+ using the paper-described initialisation, and LoRMA+ using the initialisation found in the reference implementation. All three fine-tuning configurations use Llama 3.2-3B-Instruct and are compared against the three strongest Llama prompting configurations: zero-shot (the prompting baseline), zero-shot + intent (the strongest single-metadata configuration), and few-shot + metadata (the strongest few-shot configuration). The fine-tuning configurations are matched in target modules, rank, α , base model, precision, and training setup, as described in Section 3.7.7. Parameter counts differ slightly because both LoRMA+ matrices A and B depend on d_{out} , while LoRA’s A matrix depends on d_{in} , as discussed in Section 3.7.5.

Table 4.3: Llama 3.2-3B-Instruct fine-tuning results compared to the three strongest Llama prompting configurations.

Configuration	Tr. params	Macro F1	Acc.	P(1)	R(1)	Conf. (TN, FP, FN, TP)
Majority baseline (always 0)	–	0.4732	0.898	0.000	0.000	(1068, 0, 121, 0)
TF-IDF + Logistic Regression	–	0.7161	0.851	0.387	0.793	(916, 152, 25, 96)
Llama zero-shot	–	0.3948	0.426	0.149	0.984	(387, 681, 2, 119)
Llama zero-shot + intent	–	0.4367	0.489	0.156	0.909	(471, 597, 11, 110)
Llama few-shot + meta	–	0.4201	0.458	0.158	1.000	(423, 645, 0, 121)
Llama + LoRA	9.18M	0.7437	0.907	0.542	0.537	(1013, 55, 56, 65)
Llama + LoRMA+ (paper init)	7.34M	0.7285	0.903	0.526	0.496	(1014, 54, 61, 60)
Llama + LoRMA+ (code init)	7.34M	0.5564	0.893	0.406	0.107	(1049, 19, 108, 13)

4.3.1 Implications

These results support the LoRMA+ paper’s stated initialisation convention. Under the paper-described convention, LoRMA+ performs competitively with LoRA on this task, despite using fewer trainable parameters. At the same time, the results show that the LoRMA+ paper description and its reference implementation use different initialisation schemes, and that the difference leads to a substantial gap in performance.

Both LoRMA+ initialisation schemes begin from $W' = W_0$, so the starting function is identical. The difference is whether A or B is randomly initialised at the start of training. Under the paper convention, A starts with Kaiming-initialised values and B starts at zero. This keeps the model unchanged at initialisation while allowing B to learn how to use the features produced by A , as in standard LoRA. Under the reference-implementation convention, B is Kaiming-initialised and A starts at zero. Once A begins to learn, its output is immediately routed through a randomly initialised B , which appears to substantially harm early optimisation on this task.

The 0.1721 macro F1 gap suggests that the initialisation scheme is a consequential design choice rather than an implementation detail. Future implementations using the LoRMA+ reference implementation should therefore check the initialisation block against the paper description before training, or evaluate both conventions explicitly.

Chapter 5

Discussion

5.1 Summary of Key Findings

Four findings emerge from the experimental results.

First, parameter-efficient fine-tuning is the largest observed intervention. LoRA fine-tuning on Llama 3.2-3B reaches a macro F1 score of 0.7437, compared with 0.4367 for the best prompting configuration on the same base model and 0.6263 for the best prompting configuration across all four models. The shift from prompting to fine-tuning produces a gain of approximately 0.12 to 0.31 macro F1, larger than any difference observed between prompting strategies or model choices within the prompting-only experiments. Importantly, fine-tuning is also the only intervention that exceeds the TF-IDF and logistic regression baseline (0.7161 macro F1); no prompting configuration across any of the four models exceeded the classical baseline.

Second, the LoRMA+ paper and its reference implementation disagree on initialisation, and the disagreement has a substantial empirical effect. The paper text prescribes A Kaiming-uniform and $B = 0$, matching the standard LoRA convention. The reference implementation instead declares $A = 0$ and applies Kaiming-uniform initialisation to B . Under matched hyperparameters, the paper-described initialisation produces 0.7285 macro F1 (close to LoRA’s 0.7437), while the reference-implementation initialisation produces only 0.5564 and detects just 13 of the 121 positive examples. Both schemes start from the same function $W' = W_0$, since $BAW_0 = 0$ at initialisation in both cases; the difference is which low-rank factor carries the random component during the first optimisation steps. Under the paper convention, learning begins by updating the output-side matrix from a fixed random projection through A , closely

mirroring standard LoRA. Under the reference-implementation convention, early updates to A are immediately routed through a randomly initialised output-side matrix, which appears to harm optimisation on this task. For practitioners adopting LoRMA+ from the public reference implementation, the initialisation block should be checked against the paper description before training.

Third, LoRA’s class-1 precision of 0.542 is the most important number for any deployment discussion. It is substantially better than the prompting configurations, which generally achieve class-1 precision between 0.13 and 0.27 (the dominant failure mode in prompting being over-flagging, high class-1 recall combined with low class-1 precision). But it remains far below the level required for fully automated moderation. A practical system would therefore be more appropriate as a tool that prioritises conversations for human review than as an autonomous decision-maker.

Fourth, exposing the dataset’s labelling metadata to the model in the prompt is not the same as making the model use it. The dataset’s conversation-level label is closely tied to three numerical statistics: peerness, intent-to-harm, and aggressive-message count (Section 2.2). The metadata-augmented prompts expose these values directly, yet the strongest metadata prompting configuration reaches only 0.6263 macro F1, while the fine-tuned LoRA model reaches 0.7437 using conversation text alone. Metadata augmentation helps only some models and the effect is modest: Gemma and Qwen improve when all three metadata fields are included, while Llama and Mistral do not. This suggests that current instruction-tuned LLMs may struggle to infer and apply implicit numerical decision boundaries from prompt-provided metadata, even when those metadata are highly informative.

5.2 Comparison with Related Work

Direct comparison with published results on this dataset is constrained because no peer-reviewed evaluation of modern LLMs on the conversation-level task was identified during the literature review. The papers by Ejaz et al. Ejaz et al. (2024b,c) introduce the dataset and its annotation framework but do not benchmark modern instruction-tuned LLMs, prompt strategies, LoRA fine-tuning, or LoRMA+ fine-tuning on the same splits.

The LoRMA+ paper-versus-reference-implementation discrepancy reported in Section 3.7.6 is a

distinctive contribution of this project. The LoRMA+ paper describes an initialisation convention in which A is randomly initialised and $B = 0$, while the public reference implementation uses the opposite convention. The paper does not report a controlled ablation of these two choices. This project shows that the difference is empirically consequential on the task: the paper-described initialisation reaches 0.7285 macro F1 while the reference-implementation initialisation reaches only 0.5564. This contributes a reproducibility-focused observation to the emerging literature on LoRMA+ and PEFT method implementation.

5.3 Limitations

Single seed and single hyperparameter setting. All three fine-tuning configurations were trained for 2 epochs with rank $r = 16$, scaling parameter $\alpha = 32$, learning rate 1×10^{-4} , and a single random seed. The 0.0152 macro F1 gap between LoRA and paper-initialised LoRMA+ is small enough that it may fall within normal run-to-run variance; reporting results averaged over multiple seeds would make this comparison more defensible. A modest hyperparameter sweep over rank, learning rate, dropout, and training duration could also change the ordering. The 0.1721 macro F1 gap between paper-init and code-init LoRMA+ is much larger and less likely to be explained by random variation alone, but multi-seed confirmation would still strengthen the conclusion.

Only Llama was fine-tuned. The prompting evaluation covers four models, but the fine-tuning evaluation covers only Llama 3.2-3B-Instruct. This project therefore does not test whether the three-way pattern ($\text{LoRA} \approx \text{paper-init LoRMA+} \gg \text{code-init LoRMA+}$) holds for Mistral, Qwen, or Gemma. The initialisation-scheme finding may depend on model scale: larger base models may absorb the weaker initialisation more effectively, or may show the same collapse. The choice of Llama was a memory constraint rather than a methodological preference. Fine-tuning Mistral, Qwen, or Gemma on the available 20 GB GPU would require weight quantisation, introducing an additional implementation variable into the comparison. A replication on larger hardware could fine-tune the larger models in bfloat16 and test whether the initialisation effect generalises.

Fixed few-shot demonstrations. All few-shot results use the same four demonstrations. Few-shot prompting is sensitive to example choice, ordering, and class balance, so different demonstrations could change the absolute scores and possibly the ranking between models. This

is especially relevant because the demonstration set is balanced while the test set is strongly imbalanced.

No free-form rationale generation. The prompt constrains the model to output a single character. This improves parsing reliability and keeps inference consistent across models, but rules out prompting strategies where the model generates an intermediate rationale before producing the final label. Such strategies might improve decision quality, although they would also increase parsing complexity and inference cost.

No class re-weighting or oversampling. Neither LoRA nor LoRMA+ used class re-weighting, focal loss, or oversampling. With a roughly 9:1 class imbalance, the positive-class training signal is relatively sparse. Methods that up-weight the minority class could improve class-1 recall or macro F1 and might change the relative ranking between LoRA and LoRMA+.

One dataset with dataset-specific label structure. The evaluation is limited to one conversation-level cyberbullying dataset. Its labels are closely tied to the metadata fields peerness, intent-to-harm, and aggressive-message count, making the task partly shaped by the dataset’s annotation and aggregation process. The strong gain from fine-tuning over prompting may therefore partly reflect the model learning this dataset-specific decision structure. On more naturalistic datasets, where labels are noisier or based on broader human judgement, the gain may be smaller or take a different form. The results should not be assumed to transfer to other conversation-level datasets, platforms, demographic groups, or languages without further evaluation.

5.4 Future Work

Several extensions follow from the limitations above.

First, the fine-tuning experiments should be repeated across multiple random seeds. This would provide error bars for the small gap between LoRA and paper-initialised LoRMA+, and would also test whether the weaker performance of code-initialised LoRMA+ is stable rather than seed-specific.

Second, the fine-tuning protocol should be applied to the larger base models evaluated in prompting, including Mistral 7B, Qwen 3.5-9B, and, where hardware permits, Gemma 3-27B. This would test whether the observed LoRMA+ initialisation effect generalises across model scale

and architecture.

Third, a modest hyperparameter sweep should be performed for both LoRA and LoRMA+. Varying rank, α , learning rate, dropout, and training duration would determine whether the LoRA-versus-LoRMA+ ranking is robust or dependent on the specific configuration used here.

Fourth, the evaluation should be extended to at least one additional conversation-level cyberbullying or abuse dataset. This would test whether the fine-tuning gains observed here transfer beyond the specific annotation and metadata structure .

Finally, the demographic fields excluded from prompting in this project provide a natural starting point for a follow-up study on fairness and disparate impact. Such work could examine whether model errors differ across user age, gender, or grade groups, and whether fine-tuning amplifies or reduces such disparities.

5.5 Personal Reflection

The most instructive part of this project was the implementation work that is not fully captured by the headline results. Reproducing and adapting LoRMA+ required moving beyond simply applying published methods without examining their implementation detail and instead understanding how the mathematical update, parameter shapes, module replacement, initialisation, training loop, and checkpointing procedure interact in practice.

The most important lesson came from comparing the LoRMA+ paper text with its public reference implementation. The paper describes one initialisation convention; the reference implementation uses the opposite. Identifying this discrepancy required reading both the method description and the implementation carefully, then designing a controlled ablation to test whether the difference mattered empirically. This turned what initially appeared to be an implementation detail into one of the main findings of the project, and it reinforced a broader lesson: applying modern machine learning methods is not only about running models and reporting scores. A large part of the work lies in making methods reproducible, checking that code matches theory, and understanding how small implementation choices can have large experimental effects.

Chapter 6

Legal, Social, Ethical and Professional Issues

6.1 Ethical Approval

Ethical approval for this project was obtained through the University of Surrey’s self-assessment process. The project uses a pre-existing, publicly available research dataset and does not involve the collection of new data from human participants. No interaction with human subjects took place.

The project processes only the fields already present in the published dataset. No attempt was made to identify individuals, contact users, link records to external sources, or infer additional personal information beyond the variables provided for the research task. The ethical self-assessment was reviewed and signed off by the project supervisor before 31 January 2026.

6.2 Data Ethics and Confidentiality

The dataset is licensed under Creative Commons Attribution 4.0 Creative Commons (2013). As established in Section 2.2, the dataset is semi-synthetic: the user pool is procedurally generated and the user-pair contexts are constructed through synthetic pairing, while the message text is drawn from public corpora Ejaz et al. (2024c). This reduces, but does not eliminate, the ethical risks associated with the project.

The synthetic construction means that the dataset does not preserve real private user-pair relationships. However, residual risks remain because the message text ultimately originates from real public social-media data. For this reason, the project treats the dataset as potentially sensitive even though it is publicly available. No attempt was made to re-identify individuals, link records to external sources, reconstruct original conversations, or infer additional personal information from the dataset.

All processing was performed on University of Surrey infrastructure. The report does not reproduce verbatim conversations from the dataset. Where examples are needed for explanation, they are paraphrased and lightly redacted rather than copied directly.

The trained adapters also reduce storage risk compared with full fine-tuning, because only a small number of task-specific parameters are saved while the base model remains frozen. The primary LoRA adapter contains approximately 9.18M trainable parameters, while each LoRMA+ adapter contains approximately 7.34M trainable parameters. These adapters are not designed to store or reproduce training examples verbatim, although memorisation cannot be ruled out entirely. For this reason, the project avoids releasing or presenting generated outputs that reproduce dataset conversations.

In line with the Data Protection Act 2018 UK Parliament (2018) and the UK General Data Protection Regulation Information Commissioner’s Office (2026), the project follows a data-minimisation approach: only the fields required for classification, metadata ablation, training, and evaluation are processed, and no additional personal data is collected.

6.3 Social Responsibility and Dual Use

Automated cyberbullying detection is dual-use technology. When used carefully, it can help protect users, particularly young people and people from marginalised communities, from sustained online harm at a scale that human moderators alone may struggle to match. When used poorly, it can suppress legitimate speech, disproportionately flag communities whose dialects or communication styles differ from the training distribution, and provide platform operators with a non-transparent tool for automated censorship.

False positives can silence legitimate users. The LoRA fine-tuned model’s class-1 precision is 0.542. Of every 100 conversations flagged as cyberbullying, approximately 46 are not

cyberbullying according to the dataset’s label definition. In any real deployment, this error would need to be handled through human review; otherwise the system would risk wrongful moderation of legitimate users.

False negatives leave victims unprotected. The same model’s class-1 recall is 0.537. Of every 100 actual cyberbullying conversations, approximately 46 are missed. A system used as the sole line of defence would therefore fail to catch nearly half of the cases it is intended to identify.

Distributional bias is a serious concern. The models evaluated in this project are trained on large-scale web text and may reflect the biases of that data. Applied to conversations from underrepresented communities, dialects, age groups, or cultural contexts, they may systematically over-flag or under-flag content. The dataset’s labels also reflect a particular annotation and aggregation process, not the judgement of all affected users.

Adversarial evasion is possible. A classifier with observable outputs can be probed by adversaries, who may iteratively modify their messages until they are no longer flagged. A cyberbullying detector does not eliminate cyberbullying; at best, it can increase the cost of obvious or unsophisticated abuse and support human moderation workflows.

For these reasons, any real-world deployment of this project’s outputs would require significant additional safeguards: human review of flagged content, calibration to the specific community where the system is used, ongoing checks for unequal treatment of different communities, transparency to users, and an appeals process for contested moderation decisions. The outputs of this project should therefore be understood as a research contribution, not as a deployable moderation system.

6.4 BCS Code of Conduct

The project follows the British Computer Society’s Code of Conduct BCS, The Chartered Institute for IT (2022). The dual-use risks discussed in Section 6.3 are made explicit, and the best-performing model is framed as a research evaluation rather than a deployment-ready system, addressing the public interest. Experimental settings, methods, metrics, and limitations are reported transparently, including negative results such as the reference-implementation LoRMA+ initialisation and several few-shot regressions. The project was supervised by the Department

of Computer Science at the University of Surrey, with ethical approval obtained through the self-assessment process. The methodology and code are documented to support reproduction and extension, and the LoRMA+ implementation is described with reference to the original paper, the public reference implementation, and the discrepancy between the two.

6.5 Computer Misuse Act

The Computer Misuse Act 1990 UK Government (1990) governs unauthorised access to computer systems. This project does not involve unauthorised access, interference with systems, malware development, credential misuse, or exploitation of third-party services.

All compute resources used in the project were accessed through a personal University of Surrey account with appropriate permissions. No penetration testing, vulnerability analysis, or adversarial probing of third-party systems was performed. The dataset was downloaded only from the publication channels designated by its creators, and all software dependencies were obtained from legitimate public repositories and university-provided infrastructure.

Any adversarial considerations discussed in this report relate to the behaviour and limitations of cyberbullying classifiers, not to attempts to compromise external computer systems.

6.6 United Nations Sustainable Development Goals

The project aligns with several United Nations Sustainable Development Goals United Nations (2015). SDG 16 (Peace, Justice and Strong Institutions) is most directly relevant: automated detection of sustained targeted online abuse can support safer online environments and assist platform conduct policies, provided systems remain transparent, auditable, and subject to human review (Section 6.3). SDG 10 (Reduced Inequalities) is also relevant because cyberbullying and online harassment can disproportionately affect marginalised groups; a reliable detection system could help protect users more likely to be targeted, but a biased model could worsen inequality by over-flagging underrepresented communities or by failing to detect abuse directed at them, making fairness evaluation essential. Finally, SDG 3 (Good Health and Well-being) is relevant because cyberbullying is associated with negative mental-health outcomes among young people; reducing exposure to sustained abuse may contribute indirectly to mental-health

outcomes, although automated detection alone cannot provide support, counselling, or safeguarding.

6.7 Intellectual Property

This project uses several external assets, each within the terms of its stated licence. The dataset, accessed through the `surrey-nlp/Cyberbullying-Detection-CB2` redistribution NLP-AI Group, University of Surrey (2024), is licensed under Creative Commons Attribution 4.0 Creative Commons (2013). Appropriate attribution is provided in Section 2.2.

The open-weight language models used in this project are governed by their respective model licences or terms of use. Llama 3.2-3B-Instruct is used under the Llama 3.2 Community Licence; Mistral 7B-Instruct-v0.3 and Qwen 3.5-9B are used under the licences stated in their Hugging Face model cards; and Gemma 3-27B is used under the Gemma Terms of Use. All four models are used for academic research and evaluation within the scope permitted by their respective licences.

The software libraries used in the implementation are open-source tools. Hugging Face Transformers Wolf et al. (2020), PEFT Mangrulkar et al. (2022), TRL von Werra et al. (2020), bitsandbytes Dettmers (2022), scikit-learn Pedregosa et al. (2011), and `llama.cpp` Gerganov (2023) are used under their respective open-source licences. The LoRMA+ reference implementation Bihany et al. (2025b) is publicly available and was used as a reference for understanding the method. The LoRMA+ implementation in this project was written as a custom Hugging Face-compatible module following the mathematical formulation of the method. Where the reference implementation informed design decisions, this is acknowledged in the methodology (Section 3.7.1).

All third-party datasets, model weights, libraries, and papers are cited in the report. No proprietary code, private datasets, or unauthorised assets are included in the submitted project.

Chapter 7

Conclusion

This project set out to investigate whether large language models can perform conversation-level cyberbullying detection effectively, and to characterise the conditions under which they succeed or fail. Four open-weight instruction-tuned models: Llama 3.2-3B, Mistral 7B, Qwen 3.5-9B, and Gemma 3-27B were evaluated on the conversation-level cyberbullying dataset across zero-shot prompting, few-shot prompting, metadata-augmented prompting, and parameter-efficient fine-tuning. Under matched hyperparameters, LoRA fine-tuning on Llama 3.2-3B-Instruct reached a macro F1 score of 0.7437, substantially exceeding the strongest prompting configuration (Gemma 3-27B with all three metadata fields, at 0.6263) and the strongest Llama prompting baseline (0.4367).

Three findings stand out. First, the controlled LoRA-versus-LoRMA+ comparison revealed a discrepancy between the LoRMA+ paper text and its public reference implementation: the two specify opposite initialisation conventions, and the difference produces a 0.1721 macro F1 gap between configurations that are otherwise identical (paper-initialised LoRMA+ reaches 0.7285, reference-implementation-initialised LoRMA+ reaches only 0.5564). This identifies the initialisation block as a consequential design choice that future implementations should verify against the paper before training. Second, parameter-efficient fine-tuning is a substantially larger intervention than any prompting strategy explored: the gain from prompting to fine-tuning on the same base model is approximately 0.31 macro F1, larger than any difference observed within the prompting-only experiments. Third, a dominant failure was over-flagging: they often achieved high class-1 recall but low class-1 precision, meaning that many non-bullying conversations were incorrectly classified as bullying. Fine-tuning substantially reduced this issue.

The wider lesson is that current open-weight LLMs are useful but not sufficient for conversation-level cyberbullying detection. They are useful because zero-shot prompting on Mistral, Qwen, and Gemma produces above-baseline performance, and because LoRA fine-tuning on Llama 3.2-3B substantially improves on all prompting configurations. They are not sufficient because the best fine-tuned model achieves class-1 precision of 0.542 and class-1 recall of 0.537. This is strong enough to support a decision-support tool for human moderators, but not strong enough to justify autonomous moderation. A real deployment would require human review of flagged content, calibration to the target community, ongoing audit for bias, and a transparent appeals process.

The main technical contribution is the controlled LoRA-versus-LoRMA+ comparison and the identification of the initialisation discrepancy that emerged from it. The project also contributes a parameterised experimental framework of approximately 700 lines that replaces what would otherwise have been many near-duplicate scripts, a custom Hugging Face-compatible LoRMA+ implementation applied to Llama 3.2-3B-Instruct, a controlled metadata-augmentation ablation across the three social-context fields, and an analysis of prompting failure modes under class imbalance. The most natural extensions are to repeat the fine-tuning experiments across multiple random seeds, to apply the protocol to larger base models such as Mistral 7B, Qwen 3.5-9B, and Gemma 3-27B, and to evaluate whether the LoRMA+ initialisation effect generalises across datasets and tasks.

Appendix A

Supplementary Material

A.1 Base Prompt

The following prompt was used as the shared system prompt across all prompting and fine-tuning configurations.

You are a classifier for cyberbullying in online conversations.

Task:

Determine whether the following conversation should be labeled as cyberbullying.

The conversation is provided as plain text, where each new line is a separate message in chronological order.

Definition:

Cyberbullying is sustained or clearly targeted abusive behaviour directed at a specific person or group. It includes repeated harassment, degrading personal attacks, humiliation, or threats.

Label 1 only when the overall conversation shows clear evidence of targeted cyberbullying.

Label 0 if the conversation contains only:

- isolated insults
- profanity or rude language
- a brief argument or conflict
- teasing or joking without clear abusive intent
- unclear or ambiguous targeting

Rules:

- Focus on the overall conversation, not a single message.
- Repeated abuse toward the same target is strong evidence for label 1.

- A single rude or offensive message is not enough for label 1.
- If the evidence is weak, mixed, or uncertain, output 0.

Output rules:

- Output only one character: 0 or 1
- Do not explain your answer

A.2 Few-Shot Examples

The four examples used in all `few_shot*` strategies are stored in `src/framework/few_shot_examples.py` in the submitted code archive. They are not reproduced verbatim here, in line with the data-ethics and confidentiality discussion in Section 6.2.

A.3 Hyperparameter Tables

Table A.1: LoRA hyperparameters for Llama 3.2-3B-Instruct.

Hyperparameter	Value
Base model	meta-llama/Llama-3.2-3B-Instruct
Rank r	16
α	32
Scaling α/r	2
Dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj
Initialisation A	Kaiming uniform
Initialisation B	zeros
Bias	none
Task type	CAUSAL_LM
Trainable parameters	9,175,040
Base precision	bfloat16, no quantisation
Compute dtype	bfloat16
Epochs	2
Learning rate	1×10^{-4}
Per-device batch size	1
Gradient accumulation	8
Effective batch size	8
Max sequence length	6,144 tokens
Optimiser	AdamW, PyTorch
Loss	Completion-only SFT

Table A.2: LoRMA+ hyperparameters following the paper-described initialisation in Bihany et al. (2025a).

Hyperparameter	Value
Base model	<code>meta-llama/Llama-3.2-3B-Instruct</code>
Rank r	16
α	32
Dropout $\dagger$	0.0
Target modules	<code>q_proj, k_proj, v_proj, o_proj</code>
Initialisation A	Kaiming uniform
Initialisation B	zeros
Trainable parameters $\dagger$	7,340,032
Base precision	bfloat16, no quantisation
Compute dtype	bfloat16
Epochs	2
Learning rate	1×10^{-4}
Per-device batch size	1
Gradient accumulation	8
Max sequence length	6,144 tokens
Optimiser	AdamW, PyTorch
Loss	Completion-only SFT
Save strategy $\dagger$	no; manual <code>torch.save</code> after training

Table A.3: LoRMA+ hyperparameters following the initialisation used in the official LoRMA+ reference code. This is identical to the paper-init configuration except for the two initialisation rows.

Hyperparameter	Value
Base model	<code>meta-llama/Llama-3.2-3B-Instruct</code>
Rank r	16
α	32
Dropout	0.0
Target modules	<code>q_proj, k_proj, v_proj, o_proj</code>
Initialisation A †	zeros
Initialisation B †	Kaiming uniform
Trainable parameters	7,340,032
Base precision	bfloat16, no quantisation
Compute dtype	bfloat16
Epochs	2
Learning rate	1×10^{-4}
Per-device batch size	1
Gradient accumulation	8
Max sequence length	6,144 tokens
Optimiser	AdamW, PyTorch
Loss	Completion-only SFT
Save strategy	no; manual <code>torch.save</code> after training

A.4 Full Prompting and Fine-Tuning Results

Table A.4 reports the full set of experimental results on the test set: the majority-class reference point, the TF-IDF and logistic regression baseline, 32 prompting configurations, and 3 fine-tuning configurations.

Table A.4: Full per-experiment results on the test set.

Model	Strategy	Macro F1	Acc.	P(1)	R(1)	Conf. (TN, FP, FN, TP)
majority	always 0	0.4732	0.898	0.000	0.000	(1068, 0, 121, 0)
tfidf_lr	logistic regression	0.7161	0.851	0.387	0.793	(916, 152, 25, 96)
gemma	few_shot	0.5113	0.587	0.192	0.950	(583, 485, 6, 115)
gemma	few_shot_meta	0.5208	0.594	0.200	1.000	(585, 483, 0, 121)
gemma	few_shot_peer	0.5076	0.578	0.192	0.984	(568, 500, 2, 119)
gemma	zero_shot	0.5952	0.724	0.239	0.785	(766, 302, 26, 95)
gemma	zero_shot_aggressive	0.5682	0.677	0.221	0.860	(701, 367, 17, 104)
gemma	zero_shot_intent	0.6033	0.738	0.246	0.760	(786, 282, 29, 92)
gemma	zero_shot_meta	0.6263	0.749	0.271	0.868	(785, 283, 16, 105)
gemma	zero_shot_peer	0.5840	0.723	0.226	0.711	(774, 294, 35, 86)
llama	few_shot	0.3174	0.328	0.132	1.000	(269, 799, 0, 121)
llama	few_shot_meta	0.4201	0.458	0.158	1.000	(423, 645, 0, 121)
llama	few_shot_peer	0.3702	0.393	0.143	1.000	(346, 722, 0, 121)
llama	zero_shot	0.3948	0.426	0.149	0.984	(387, 681, 2, 119)
llama	zero_shot_aggressive	0.4139	0.450	0.155	0.992	(415, 653, 1, 120)
llama	zero_shot_intent	0.4367	0.489	0.156	0.909	(471, 597, 11, 110)
llama	zero_shot_meta	0.4150	0.455	0.152	0.950	(426, 642, 6, 115)
llama	zero_shot_peer	0.4027	0.437	0.150	0.975	(401, 667, 3, 118)
mistral	few_shot	0.5481	0.653	0.206	0.843	(674, 394, 19, 102)
mistral	few_shot_meta	0.5458	0.627	0.215	1.000	(625, 443, 0, 121)
mistral	few_shot_peer	0.5801	0.711	0.225	0.752	(754, 314, 30, 91)
mistral	zero_shot	0.4743	0.533	0.176	0.975	(516, 552, 3, 118)
mistral	zero_shot_aggressive	0.4528	0.504	0.167	0.975	(481, 587, 3, 118)
mistral	zero_shot_intent	0.4360	0.481	0.161	0.975	(454, 614, 3, 118)
mistral	zero_shot_meta	0.4216	0.461	0.157	0.984	(429, 639, 2, 119)
mistral	zero_shot_peer	0.4435	0.491	0.164	0.975	(466, 602, 3, 118)
qwen	few_shot	0.4144	0.450	0.156	1.000	(414, 654, 0, 121)
qwen	few_shot_meta	0.5276	0.603	0.204	1.000	(596, 472, 0, 121)
qwen	few_shot_peer	0.4403	0.484	0.165	1.000	(455, 613, 0, 121)
qwen	zero_shot	0.5257	0.609	0.198	0.934	(611, 457, 8, 113)
qwen	zero_shot_aggressive	0.4904	0.553	0.185	0.992	(538, 530, 1, 120)
qwen	zero_shot_intent	0.5945	0.702	0.244	0.917	(724, 344, 10, 111)
qwen	zero_shot_meta	0.5988	0.706	0.248	0.926	(728, 340, 9, 112)
qwen	zero_shot_peer	0.5478	0.640	0.211	0.926	(649, 419, 9, 112)
llama	LoRA (FT)	0.7437	0.907	0.542	0.537	(1013, 55, 56, 65)
llama	LoRMA+ paper init (FT)	0.7285	0.903	0.526	0.496	(1014, 54, 61, 60)
llama	LoRMA+ code init (FT)	0.5564	0.893	0.406	0.107	(1049, 19, 108, 13)

Bibliography

BCS, The Chartered Institute for IT. Code of conduct. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>, 2022. Accessed: 2026-05-22.

Harsh Bihany, Shubham Patel, and Ashutosh Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10113–10133, Vienna, Austria, 2025a. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.527. URL <https://aclanthology.org/2025.findings-acl.527/>.

Harsh Bihany, Shubham Patel, and Ashutosh Modi. LoRMA reference implementation. GitHub repository, Exploration-Lab/LoRMA, 2025b. URL <https://github.com/Exploration-Lab/LoRMA>. Accessed: 14 May 2026.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

Creative Commons. Attribution 4.0 international (cc by 4.0). <https://creativecommons.org/licenses/by/4.0/>, 2013. Accessed: 2026-05-22.

Thomas Davidson, Dana Warmusley, Michael Macy, and Ingmar Weber. Automated hate speech detection and the problem of offensive language. In *Proceedings of the International AAAI Conference on Web and Social Media*, 2017.

- Tim Dettmers. bitsandbytes. <https://github.com/bitsandbytes-foundation/bitsandbytes>, 2022.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the North American Chapter of the Association for Computational Linguistics*, 2019.
- Naveed Ejaz, Salimur Choudhury, and Fakhra Razi. A comprehensive dataset for automated cyberbullying detection, 2024a.
- Naveed Ejaz, Fakhra Razi, and Salimur Choudhury. Towards comprehensive cyberbullying detection: A dataset incorporating aggressive texts, repetition, peerness, and intent to harm. *Computers in Human Behavior*, 153:108123, 2024b.
- Naveed Ejaz, Fakhra Razi, and Salimur Choudhury. A multi-faceted semi-synthetic dataset for automated cyberbullying detection. *arXiv preprint arXiv:2402.10231*, 2024c.
- Fatma Elsafoury. Cyberbullying datasets, 2020. Kaggle.
- Antigoni Maria Founta, Despoina Chatzakou, Nicolas Kourtellis, Jeremy Blackburn, Athena Vakali, and Ilias Leontiadis. Large scale crowdsourcing and characterization of twitter abusive behavior. In *Proceedings of the International AAAI Conference on Web and Social Media*, 2018.
- Gemma Team, Google DeepMind. Gemma 3 technical report. Technical report, 2025.
- Georgi Gerganov. llama.cpp. <https://github.com/ggml-org/llama.cpp>, 2023.
- Google DeepMind. Gemma-3-27B-IT. Hugging Face model card, 2025. URL <https://huggingface.co/google/gemma-3-27b-it>. Accessed: 14 May 2026.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Hugging Face. Chat templates. https://huggingface.co/docs/transformers/chat_templating, 2026a. Accessed: 2026-05-22.
- Hugging Face. Mistral model documentation. Transformers documentation, 2026b. URL https://huggingface.co/docs/transformers/en/model_doc/mistral. Accessed: 14 May 2026.
- Hugging Face. SFTTrainer. TRL documentation, 2026c. URL https://huggingface.co/docs/trl/en/sft_trainer. Accessed: 14 May 2026.
- Information Commissioner’s Office. Uk general data protection regulation (uk gdpr). <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/>, 2026. Accessed: 2026-05-22.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Ritesh Kumar, A. N. Reganti, Akshit Bhatia, and Tushar Maheshwari. Aggression-annotated corpus of hindi-english code-mixed data. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation*, 2018.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the Association for Computational Linguistics*, 2021.
- Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*, 2022.
- Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, and Sayak Paul. PEFT: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.

- Meta AI. Llama 3.2-3B-Instruct configuration file. Hugging Face model configuration, 2024a. URL <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct/blob/main/config.json>. Accessed: 14 May 2026.
- Meta AI. Llama 3.2-3B-Instruct. Hugging Face model card, 2024b. URL <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>. Accessed: 14 May 2026.
- Mistral AI. Mistral-7B-Instruct-v0.3 configuration file. Hugging Face model configuration, 2024a. URL <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3/blob/main/config.json>. Accessed: 14 May 2026.
- Mistral AI. Mistral-7B-Instruct-v0.3. Hugging Face model card, 2024b. URL <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>. Accessed: 14 May 2026.
- mradermacher. gemma-3-27b-it-GGUF. Hugging Face Hub, 2025. URL <https://huggingface.co/mradermacher/gemma-3-27b-it-GGUF>. Accessed: 14 May 2026.
- Charisse L. Nixon. Current perspectives: the impact of cyberbullying on adolescent health. *Adolescent Health, Medicine and Therapeutics*, 5:143–158, 2014. doi: 10.2147/AHMT.S36456.
- NLP-AI Group, University of Surrey. Cyberbullying-Detection-CB2, 2024. URL <https://huggingface.co/datasets/surrey-nlp/Cyberbullying-Detection-CB2>. Hugging Face Datasets: surrey-nlp/Cyberbullying-Detection-CB2.
- Dan Olweus. School bullying: Development and some important challenges. *Annual Review of Clinical Psychology*, 9:751–780, 2013. doi: 10.1146/annurev-clinpsy-050212-185516.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

- Qwen Team. Qwen3.5-9B. Hugging Face model card, 2026a. URL <https://huggingface.co/Qwen/Qwen3.5-9B>. Accessed: 14 May 2026.
- Qwen Team. Qwen3.5-9B configuration file. Hugging Face model configuration, 2026b. URL <https://huggingface.co/Qwen/Qwen3.5-9B/blob/main/config.json>. Accessed: 14 May 2026.
- Peter K. Smith, Jess Mahdavi, Manuel Carvalho, Sonja Fisher, Shanette Russell, and Neil Tippet. Cyberbullying: Its nature and impact in secondary school pupils. *Journal of Child Psychology and Psychiatry*, 49(4):376–385, 2008.
- UK Government. Computer misuse act 1990, 1990. URL <https://www.legislation.gov.uk/ukpga/1990/18/contents>.
- UK Parliament. Data protection act 2018. <https://www.legislation.gov.uk/ukpga/2018/12/contents>, 2018. Accessed: 2026-05-22.
- United Nations. The 17 sustainable development goals. <https://sdgs.un.org/goals>, 2015. Accessed: 2026-05-22.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
- Zeera Waseem and Dirk Hovy. Hateful symbols or hateful people? predictive features for hate speech detection on twitter. In *Proceedings of the NAACL Student Research Workshop*, 2016.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.